TRANSFER LEARNING FOR IMPROVED VIDEO DESCRIPTION PERFORMANCE FOR VISUALLY IMPAIRED AND BLIND

A thesis presented to the faculty of
San Francisco State University
In partial fulfillment of
The Requirements for
The Degree

Master of Science
In
Computer Science

by

Andrew Taylor Scott

San Francisco, California

May 2020

CERTIFICATION OF APPROVAL

I certify that I have read *TRANSFER LEARNING FOR IMPROVED VIDEO DESCRIPTION PERFORMANCE FOR VISUALLY IMPAIRED AND BLIND* by Andrew Taylor Scott and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirements for the degree: Master of Science in Computer Science at San Francisco State University.

Dr. Ilmi Yoon
Professor of Computer Science

Dr. Pooyan Fazli
Assistant Professor of Computer Science

Dr. Arno Puder
Department Chair of Computer Science

TRANSFER LEARNING FOR IMPROVED VIDEO DESCRIPTION PERFORMANCE FOR VISUALLY IMPAIRED AND BLIND

Andrew Taylor Scott
San Francisco State University
2020

This work focuses on improving the caption quality of videos for visually impaired and blind users. We develop experimental procedures and analyze results of automatically generated captions to be used to pre-seed annotations for a Human-in-the-Loop Machine Learning interface. We use transfer learning to take a pretrained neural network and fine-tune it on a domain-specific dataset. We find that expanding the vocabulary is useful and that the transferred knowledge has an impact on results. More work needs to be done to refine the captions and obtain better results by human standards.

I certify that the Abstract is a correct representation of the content of this thesis.

Chair, Thesis Committee Date

ACKNOWLEDGMENTS

I'd like to thank Dr. Ilmi Yoon for her belief in me and providing the opportunity to explore this material in-depth; Dr. Pooyan Fazli for his participation in the project and use of the People and Robots Laboratory (PeRL), and for being on the committee; Dr. Abhishek Das and Yash Kant for their expert domain knowledge in deep learning and support running experiments; The YouDescribe Group for their work on the various and parallel components of this project; The Department of Computer Science at San Francisco State University, especially Dr. Kazunori Okada for his mentorship in pattern analysis and machine intelligence. This project would not have been possible without the help of SKERI and FAIR for providing the platform and tools, the YouCookII Team for the dataset, and the Pythia/MMF Team for the framework. I am also very grateful to Blue River Technology — they are an excellent company to work for. I'd particularly like to thank Chris Padwick for the encouragement and guidance in graduate studies; The CVML Team at BRT for their enthusiasm and inspiration; and everyone at BRT who have shown me amazing support while completing the program. I'd also like to thank my Uncle, Robert Gray Tuel, for his positivity toward continuing education; my neighbor, Rachael Pressel, for letting me use her WiFi to upload datasets; and my grandmother for her love.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

Chapter 1

Introduction

The goal of this study is to improve the quality of computer generated video captions for visually impaired and blind people by using the current state of the art in deep learning. There are several approaches to this task that we shall discuss. Our focus is on transfer learning and caption quality. We take a pretrained, large artificial neural network model and fine-tune it on a domain specific dataset. This involves two key areas of machine learning: differentiable programming and natural language processing.

The core of our contributions is that we integrate the publicly available dataset of cooking videos, YooCookII[124], with the computer vision and NLP framework, Pythia[102, 103], so that we can learn a combined image and language model for the task of captioning cooking videos. Other contributions that needed to be developed in the process of this work were that we leverage cosine similarity as a way to measure caption quality and we make several Web services and utilities for captioning,

scoring, and inspecting the data. All source code is available in the author's github[1] repository.

Training a deep learning model on a large video database is computationally expensive. In order to make our problem more tractable, we focus on image captioning as a way to further our understanding of video captioning. This makes sense from a logical standpoint since videos are a sequence of images and any method that improves the quality of image captioning, it is presumed, may be extended to captioning a sequence of images. We acknowledge this approach neglects features that are in videos and which are not present in the images alone such as temporality and optical flow. We see this study as a foundation upon which we can explore and develop those more computationally expensive ideas later.

The central hypothesis of this work is that image caption quality may be improved for a specific domain by fine-tuning a pretrained model on a domain-specific dataset. We use cosine similarity as an indirect metric of caption quality and find that this method shows promise but we were unable to achieve better performance than ground-truth sentences. The work shows that extending vocabulary from the domain-specific dataset is useful. With more experimentation, we expect this line of research to be fruitful, even though our initial results here are inconclusive.

[1] https://github.com/ascott02

Chapter 2

Background of the Study

In collaboration with Smith Kettlewell Eye Research Institute[1] (SKERI) and directed by Dr. Ilmi Yoon and Dr. Pooyan Fazli from San Francisco State University, and in collaboration with Dr. Abhishek Das and Yash Kant from Georgia Tech, and with support from Facebook's FAIR group[2], a research group at SFSU worked on a multi-faceted research project to improve the quality of video captioning for visually impaired and blind people. This Masters thesis is one product of that effort and part of a larger effort, directed by Dr. Yoon, to provide human-in-the-loop machine learning accessibility to videos[24]. The larger project is involved in the YouDescribe website[3], where visually impaired people can *view* human annotated videos and a game that can be played to generate ratings of individual images and captions. This is part of a Human-in-the-Loop Machine Learning (HILML), directed

[1] https://www.ski.org
[2] https://ai.facebook.com
[3] https://youdescribe.org/

by Dr. Yoon[120]. The contributions of this thesis work involve improvements to the deep learning generated captions used to seed and prompt participants when playing the game. The descriptions and ratings can then be used to train another model for full video description generation and eventually promoted to the video annotations on the YouDescribe site.

According to the World Health Organization's Global Data on Visual Impairments, 2010 report, there are 285 million visually impaired people in the world [14]. That number includes 39 million who are blind, and 246 million with low vision. A study by the National Foundation of the Blind found there were 7.6 million people living with a visual disability in the United States, in 2016[4]. Their definition of visual disability, includes anyone who "must use alternative methods to engage in any activity that people with normal vision would do using their eyes"[4].

An international survey, from 2017, by WebAIM (Web Accessibility in Mind)[5], a non-profit group that is part of the Center for Persons with Disabilities[111], at Utah State University, found that out of 1792 participants, 75.8% are blind, 20.4% have low vision, and 75.6% rely exclusively on screen reader audio with no visual information. The same survey also showed that 40.8% of the participants felt that Web content was becoming more accessible over the past year while 59.2% think it has not changed or become less accessible, and 85.3% think that Web sites need to be made more accessible, rather than creating more assistive technologies. There is strong need for more accessibility to rich new media content.

Much of current new media is visual, in the form of videos and images. This includes Web sites like YouTube, Facebook and Twitter, where videos and images are shared to millions of users. This media is often inaccessible to a large number of people unless publishers pay to hire someone to manually annotate and describe them. Blind and visually impaired users can listen to the audio track for a video but if there is no dialog or it is visually driven content then it is difficult to understand the full content of what is occurring visually.

Video description is like a friend sitting next to you in a movie theater, whispering in your ear, the visual details they cannot see. Set and setting, who's in the scene, and what else is going on, not directly in the dialog are details that will be missed. Automating video description is then akin to making a computer whisper in your.

The YouDescribe project aims to make videos and visual content more accessible to visually impaired and blind people by providing quality descriptions of the imagery so that they can listen to the information that they are missing. Figure 2.1 shows a typical image and caption from the MS COCO dataset. COCO stands for Common Objects in COntext.

Figure 2.1: "Man riding a motor bike on a dirt road on the countryside." Example image and ground-truth caption from MS COCO 2014 validation set[67].

"A significant bottleneck in video accessibility for blind and visually impaired users is the time and cost to produce video description professionally"[120]. It could cost thousands of dollars for one video, depending on the length for professionally annotated videos[120]. Alternatives to professional description services such as MAGpie[18] and LiveDescribe[1] allow anyone to describe videos but these solutions are stand-alone software programs for do-it-yourself annotating of videos. There is a crowd-sourced application, called Be My Eyes[3], which has over two million users, and connects visually-impaired and blind users with sighted users. This works in real-time through a smart phone application but does not scale since each description is one-on-one and tailored to that experience. Informal feedback from volunteers in [120] showed that there are significant hurdles to creating quality descriptions for

videos in this way. Many of the volunteers do not have enough language knowledge or know how to provide succinctly informative and descriptive captions for videos that capture the correct visual elements.

In Yoon et al.[120], the authors illustrate the challenges of creating quality descriptions and develop a Human-in-the-Loop Machine Learning (HILML) approach to improving these captions. They develop this approach to video description to reduce the cognitive load on the describer and simultaneously improve the overall quality of captions. The approach uses pre-generated captions to prompt the describer and to guide their own descriptions. In the study, this reduced the average time to create descriptions from $\mu = 1825.45 \pm 658.51$ to $\mu = 1285.41 \pm 659.51$. See Figure 2.2. In their follow-up survey of describers, mental demand, effort and frustration were significantly lower, and 16 out of 22 describers felt the HILML was easier for them to use[120]. See figures 2.3. However, another survey question shows that there was only neutral belief that the text was accurate. See Figure 2.4. A follow-up survey of blind users also favored the HILML descriptions over Human-only descriptions. See Figure 2.5.

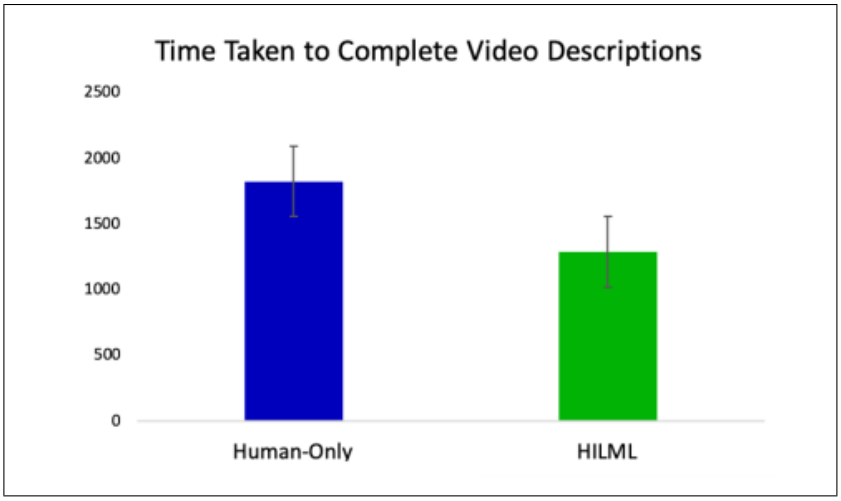

Figure 2.2: Time taken by describers using Human-Only versus HILML. Plot taken from [120].

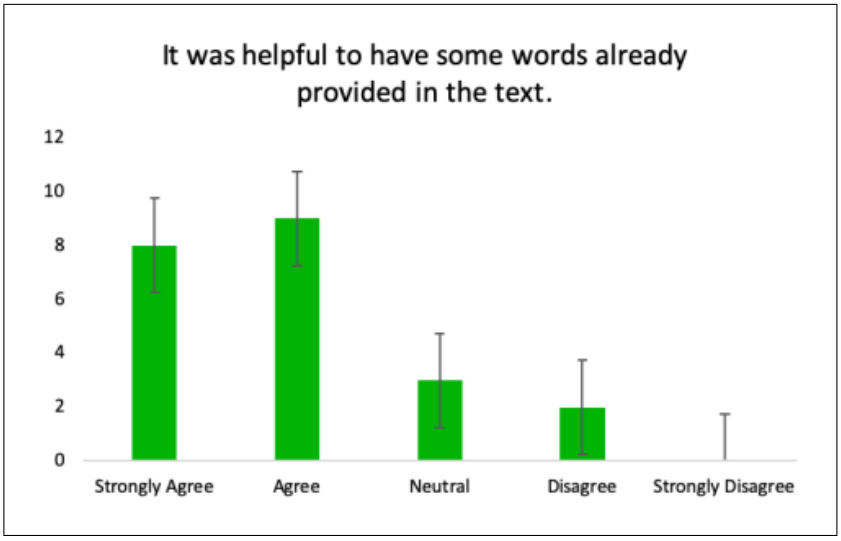

Figure 2.3: Describers who thought it was helpful to have words already provided. Plot taken from [120].

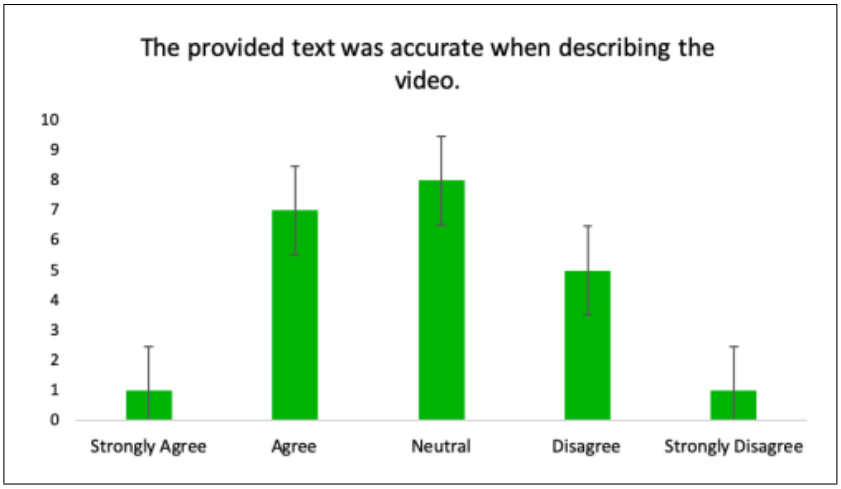

Figure 2.4: Describers who thought the provided words were accurate. Plot taken from [120].

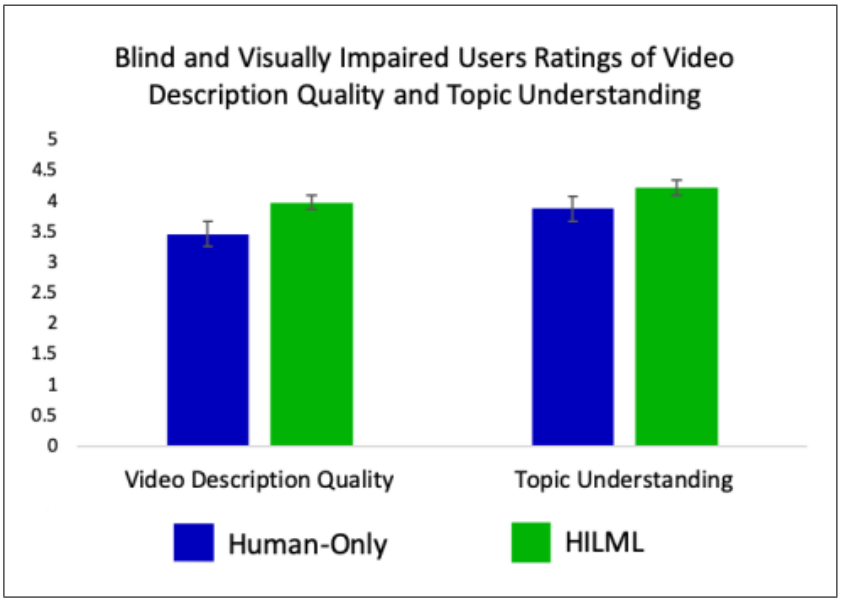

Figure 2.5: Blind users who surveyed thought that the quality of the descriptions and topic understanding were generally better HILML approach. Plot taken from [120].

A new site, YouDescribe, combines crowd-sourcing with YouTube videos. The HILML approach is currently in the process of being integrated into that site, with this work being a key component: automated caption generation. A screenshot of the YouDescribe home page is presented in Figure 2.6. They currently have 2500 sighted volunteers, and over 4000 videos[120]. The platform allows volunteers to pick a video and annotate it while viewing the video. See Figure 2.7 for a screenshot of the annotation interface.

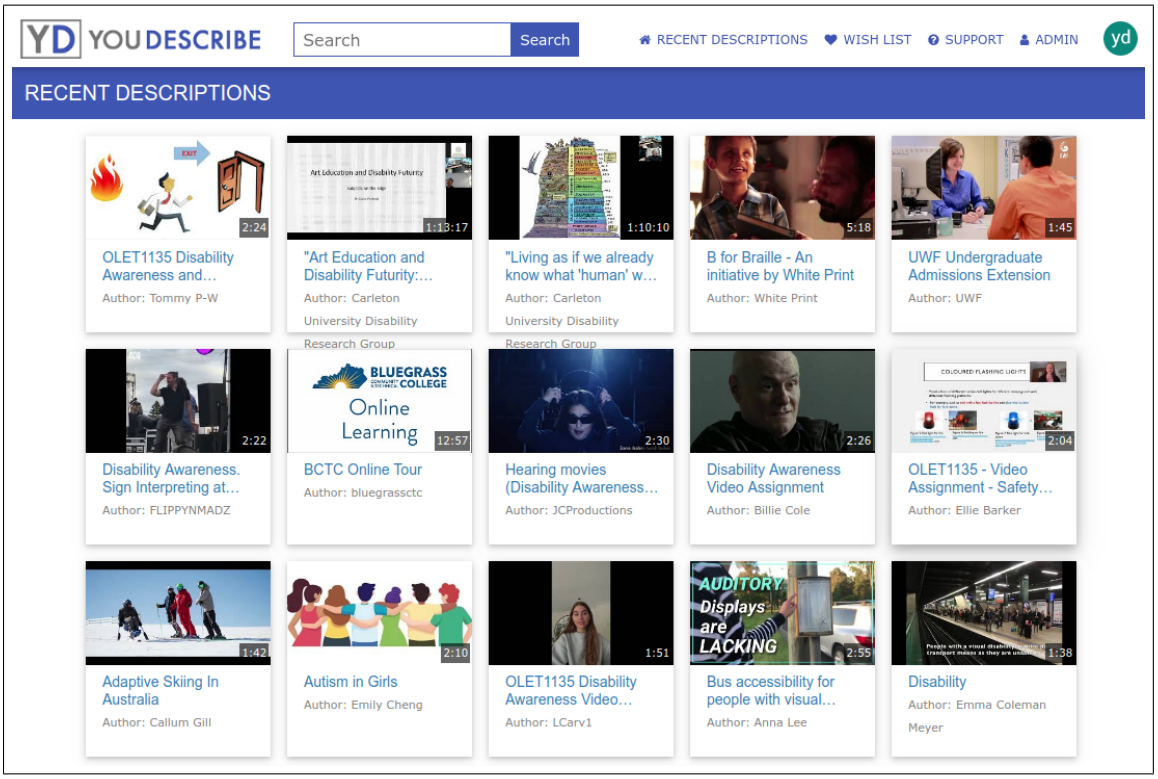

Figure 2.6: YouDescribe Home Page[2].

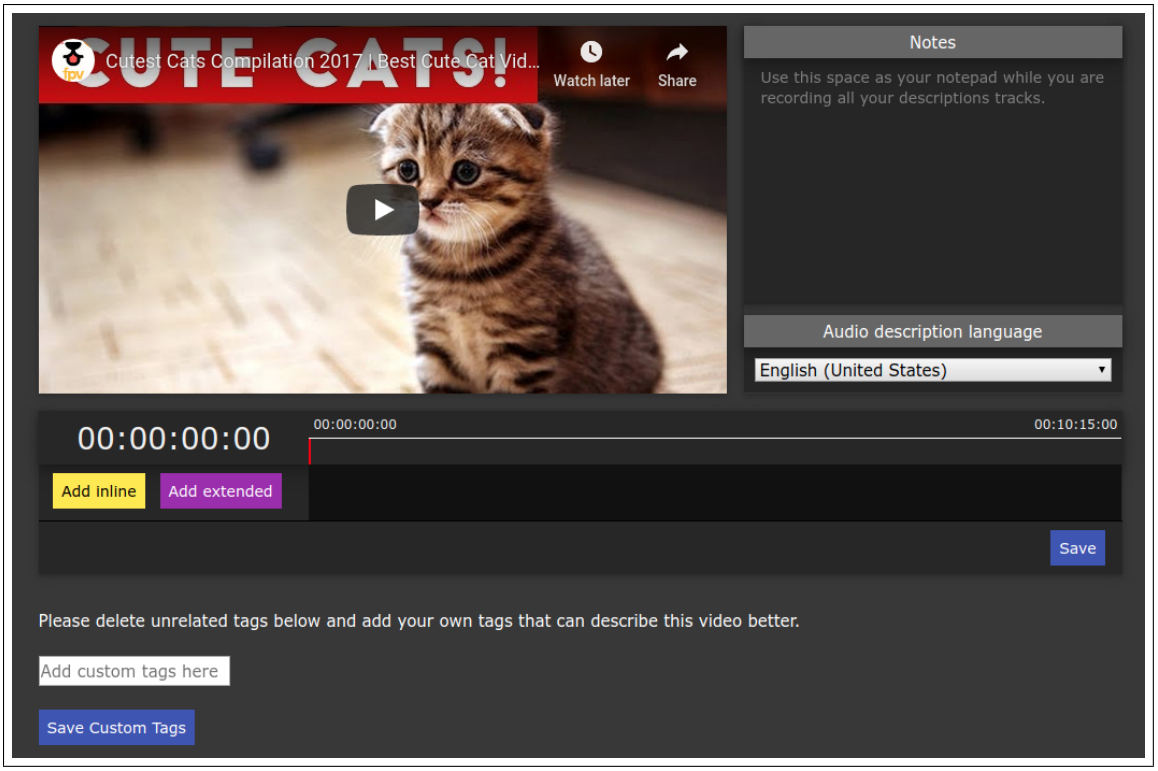

Figure 2.7: YouDescribe Describer Annotation Interface[2].

In the annotation interface a describer watches the video they are describing. They can pause, fast-forward, and rewind. And they can add manually entered annotations describing the scenes in the video at specific time indexes. Using the HILML approach, the describer is presented with a pre-generated description that can guide them while writing their own description.

This thesis is focused on improving the pre-generated captions by transferring knowledge from another dataset. The captions will be used in the HILML approach to describing videos in the YouDescribe Web site.

Chapter 3

Current State of the Art

We take a piece-wise approach to video captioning by first starting with image captioning. Image captioning is a computer vision and natural language processing problem. It requires background knowledge of both areas of research. We will review the basic ideas underlying both topics, along with transfer learning and the current state of the art.

3.1 Computer Vision

Computer vision has been well-researched in computer science and continues to be an interesting topic. It involves taking an image, preprocessing it, applying some algorithm to it, and returning the result of that algorithm. Images can be taken from any camera, a frame from a video, or any other digitized picture. Computer vision has applications in bio-medical imagery, face-detection, object identification, and agriculture, to name a few. Common tasks are classification, object detection and

segmentation. Traditional machine learning approaches to computer vision include algorithms such as linear regression, principal component analysis, and support vector machines. Current state of the art approaches to computer vision are, almost exclusively, based on convolutional neural networks because of their ability to generalize. However, there are still some current projects that are using traditional machine learning methods, such as VLFeat[113] which uses SVM features.

3.1.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are at the heart of many current deep learning computer vision algorithms. They are often referred to as the backbone network in many computer vision architectures. CNNs fall under the umbrella of deep learning and which is also a form of differentiable programming on account of the gradient decent. Deep learning algorithms are successors of early perceptrons but have many more layers and are therefore *deep*. They optimize gradient decent across each neuron. Convolutional neural networks are so named because of the convolutional operation applied to a sliding window that scans across the image. Image features to be used as input to other architectures are often extracted from the last full-connected layer. CNNs are a type of *feed-forward* network, because values are propagated forward to the next layer. CNNs also use *back-propagation*, during training. Current state of the art CNN architectures include algorithms such as AlexNet[60], VGGNet[101], ResNet[43], R-CNN[38], Fast R-CNN[37], Faster R-CNN[88], Mask

R-CNN[42], Bottom Up Top Down[16], and Transformers[112]. BuTD, Mask R-CNN and Transformers are the current state of the art.

In order to input an image into a artificial neural network, the image is usually preprocessed by resizing and value normalization, along with other augmentations, and then input as a multidimensional value set. A color JPEG image can be thought of as a matrix of tuples of three values $(red, green, blue)$. A *10 by 10* color JPEG image has the dimensions $(10, 10, 3)$. An artificial neural network that can read that image would generally need those same dimensions on the input layer. This is not an exact analog but it is similar to how most animal's eyes work. There is a grid of photoreceptor cells, the retina, that are photosensitive and convert light into signals to the brain or nervous system.

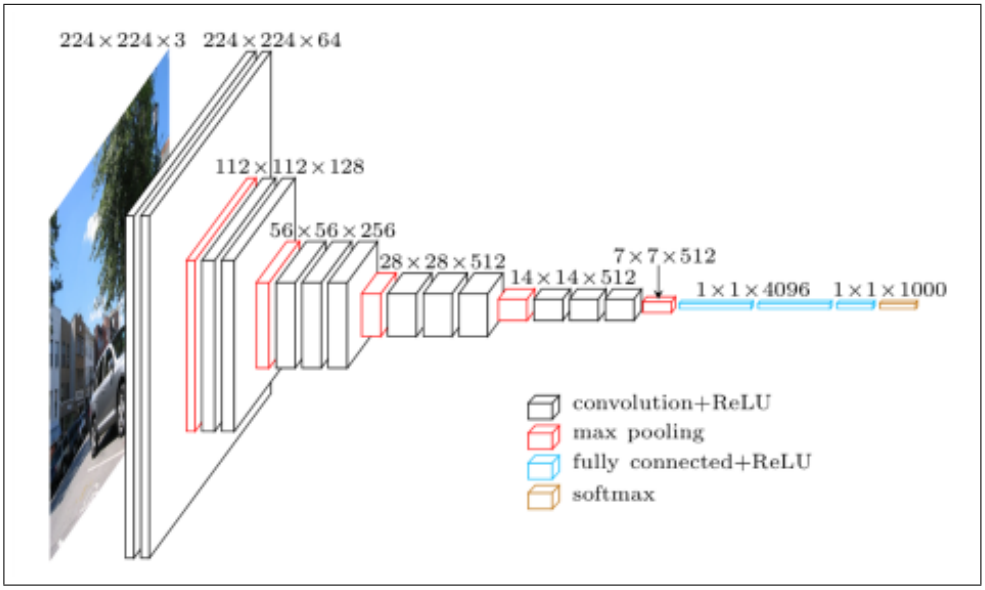

Figure 3.1: Conceptual view of VGG-16 taken from [110].

Figure 3.1 shows the conceptual diagram of the layers of a common deep learning neural network architecture. This particular architecture is of VGG-16[101]. It has 16 convolutional and full-connected layers. The fully connected layers are an easy way to have non-linear mappings from inputs to outputs. The pooling layers reduce the dimensionality at each pooling layer.

The VGG-16 architecture starts with an input image of size $(224 \times 224 \times 3)$. The image is run through two layers of convolution filters and ReLU, then max pooling reduces the dimensions to $(112 \times 112 \times 64)$. ReLU stands for Rectified Linear Unit, and clips values between 0 and 1. If a value is less than 0 it becomes 0. If the value is greater than 0, then the value it kept. See figure 3.2. The features are then passed through three more convolutional layers and max pooling again, reducing the dimensions to $(56 \times 56 \times 256)$. Then three more convolutional layers and max pooling again, reducing dimensions to $(14 \times 14 \times 512)$. Max pooling reduces the dimensions one more time to $(7 \times 7 \times 512)$ and then the features are run through three full-connected layers and softmax to make the classification.

Figure 3.2: Rectified Linear Unit (ReLU) activation function. [7].

Figure 3.3 shows the `model.summary()` of the architecture, with the dimension of each layer and the number of parameters. There are over 134 million parameters in this model.

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 224, 224, 64)	1792
conv2d_2 (Conv2D)	(None, 224, 224, 64)	36928
max_pooling2d_1 (MaxPooling2	(None, 112, 112, 64)	0
conv2d_3 (Conv2D)	(None, 112, 112, 128)	73856
conv2d_4 (Conv2D)	(None, 112, 112, 128)	147584
max_pooling2d_2 (MaxPooling2	(None, 56, 56, 128)	0
conv2d_5 (Conv2D)	(None, 56, 56, 256)	295168
conv2d_6 (Conv2D)	(None, 56, 56, 256)	590080
conv2d_7 (Conv2D)	(None, 56, 56, 256)	590080
max_pooling2d_3 (MaxPooling2	(None, 28, 28, 256)	0
conv2d_8 (Conv2D)	(None, 28, 28, 512)	1180160
conv2d_9 (Conv2D)	(None, 28, 28, 512)	2359808
conv2d_10 (Conv2D)	(None, 28, 28, 512)	2359808
max_pooling2d_4 (MaxPooling2	(None, 14, 14, 512)	0
conv2d_11 (Conv2D)	(None, 14, 14, 512)	2359808
conv2d_12 (Conv2D)	(None, 14, 14, 512)	2359808
conv2d_13 (Conv2D)	(None, 14, 14, 512)	2359808
max_pooling2d_5 (MaxPooling2	(None, 7, 7, 512)	0
flatten_1 (Flatten)	(None, 25088)	0
dense_1 (Dense)	(None, 4096)	102764544
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 4096)	16781312
dropout_2 (Dropout)	(None, 4096)	0
dense_3 (Dense)	(None, 2)	8194

Total params: 134,268,738
Trainable params: 134,268,738
Non-trainable params: 0

Figure 3.3: Model dimensions and parameters for VGG-16 [110].

Convolution involves moving a sliding kernel filter across the image, similar to morphological operators. Each element in the image is added to the local neighbors weighted by the kernel filter. Figure 3.4 shows a simple conceptual diagram of the convolutional operation.

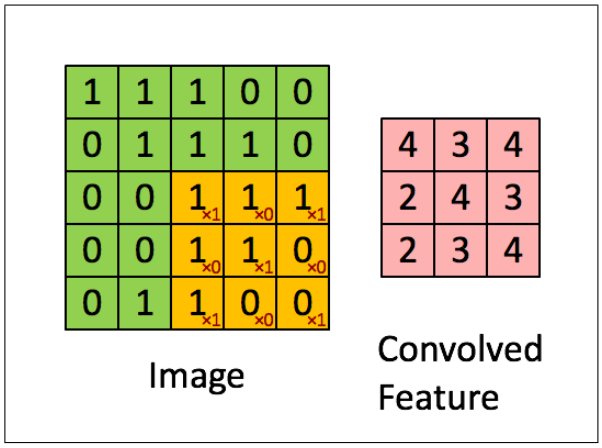

Figure 3.4: Visualization of convolution operation taken from [84].

Figure 3.5 shows max pooling and average pooling, respectively. Pooling also uses a sliding window across the image.

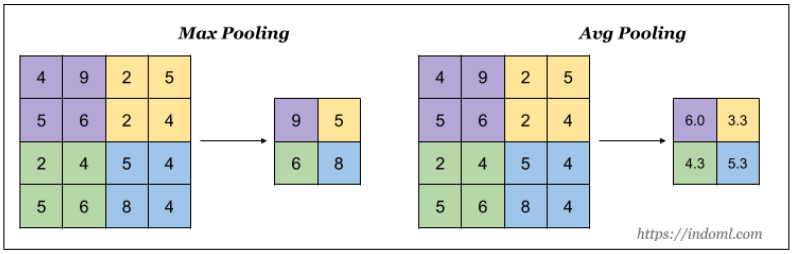

Figure 3.5: Visualization of max and average pooling operation taken from [84].

3.1.2 Transfer Learning

In machine learning, the concept of transfer learning involves taking a model from one one domain and applying it to another. Transfer learning has become popular due the availability of very large models and the time it takes to train them. The idea, in the most basic, is than train a model completely from scratch, we can bootstrap the model by loading a pretrained model first. This has become very common with large datasets such as MS COCO[67] and Imagenet[29]. These models can take a very long time to train. So, many open source projects have taken to publishing the pretrained model publicly, where others can load that model and use it without having to train it themselves. People also take these models and *fine-tune* them by training with more data on top of the pretrained model. When the new data comes from an entirely different domain, this is considered transfer learning. We are transferring knowledge from one domain and applying it to another.

For our project, we are interested in improving the caption quality of image captions for cooking videos. In order to do this, we want to preload a large dataset such as COCO, and then fine-tune with another dataset from the target domain, cooking.

3.2 Natural Language Processing

Natural Language Processing (NLP) applications range from virtual assistants and chatbots to language translation, recommendation systems and sentiment analysis. In the most basic sense, NLP involves getting a computer to understand human language, or at least give the appearance of understanding. The term *understand* in that sentence is one of philosophical debate but in our use we mean that the computer can process human language in a useful manor. For example, if we wanted to create a conversational chatbot, it would not be useful if it generated sentences composed of random words. The broader field of general artificial intelligence is concerned with whether or not the computer understood what it read. For information about that, please see Searle's Chinese Room example[95]. By contrast, current NLP is more concerned with whether the computer processes and outputs human language in a way that is meaningful to humans. In this work, we use human understanding as a common-sense measure of quality at the most basic level — a caption or description is considered *good* if it is consistent with human understanding.

The book, Natural Language Processing with Tensorflow, by Thushan Ganegedara lists the most common tasks in NLP as: tokenization, word-sense disambiguation, named entity recognition, part-of-speech tagging, sentence/synopsis classification, language generation, question answering, and machine translation[34]. Looking at these one at a time. Tokenization is the task of decomposing a corpus of text, such as a novel, into a atomic units, such as word or phrases. Word-sense disambiguation

(WSD) is the task of distinguishing words that are spelled the same way but have different meanings in different contexts. It is very important to the question answering task. Named entity recognition (NER) is the process of identify defined entities in a corpus for information retrieval and knowledge representation. The part-of-speech (PoS) tagging task involves identifying nouns, verbs, adjectives, adverbs, or even even finer details about the sentence. Sentence/synopsis classification involves assigning a label to a a sentence or other collection of words. Sentiment analysis is a common use of sentence/synopsis classification: this tweet is positive or negative. Language generation involves generating new words, given previous words and often involves using an artificial neural network trained on similar data. For example, to generate news articles, you can use a corpora of past news articles as training data. Question answering (QA) is the task of responding to specific questions about the domain. QA is notoriously difficult and necessary for chatbots and virtual assistants. Machine translation (MT) is the task of converting one language, such as English, into another language, such as Spanish. This is also a very difficult task since not all languages do not have a one-to-one correspondence between words and sentences. Our task of captioning images and videos, is primarily one of language generation but also involves aspects of all of these tasks.

3.2.1 Word Embeddings

In order to vectorize the words and n-grams in a sentence, words and n-grams are embedded in a shared semantic space, by one-hot-encoding each word and assigning it to a vector. A sentence in the embedding space is then composed of several word vectors. The word2vec[78] library is one of the first publicly available word embeddings to be reused for NLP projects. Another, newer embedding methodology is GloVe[83]. Figure 3.6 shows a visualization of both embeddings.

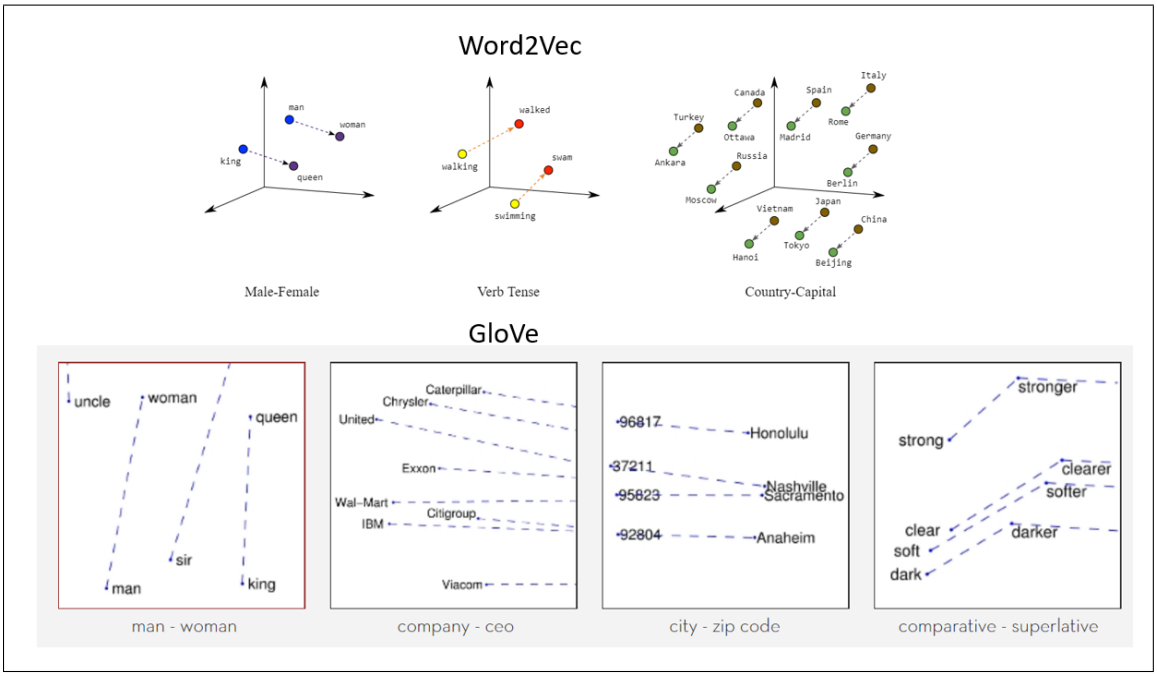

Figure 3.6: Visualization of *word2vec* and *GloVe* embeddings taken from [53].

Word2vec and GloVe both try to bring words similar meanings closer together and push dissimilar words further apart. GloVe uses "global matrix factorization

methods like latent semantic analysis (LSA) for generating low-dimensional word representations"[53] and a context window which makes a smaller, yet more meaningful, dimensional space than word2vec. It is not a perfect space though — some meaning is still lost.

The current state of the art architectures for word embeddings are now using BERT, which stands for Bidirectional Encoder Representations from Transformers[31]. BERT uses multiple transformer attention heads for bidirectional encoding.

3.2.2 Algorithms

Recurrent Neural Networks (RNNs) are another type of neural network that does not have any convolutional layers. RNNs are particular good at sequence prediction and therefore used in NLP applications such as language generation. RNNs are a type of neural network "where connections between nodes form a directed graph along a temporal sequence"[12] and "allows it to exhibit a temporal dynamic behavior"[12]. There are several types of RNNs, the most common are Long-Short Term Memory (LSTM), Gate Recurrent Units (GRUs), and Bi-LSTMs. More recently, Transformers and other CNN-based approaches have started to find applications where RNNs were previously used because CNNs can be trained in parallel, whereas RNNs are serial in nature.

The basic idea with RNNs is that each node in the network not only feeds values forward but also feeds values back to itself from the previous pass. This gives the

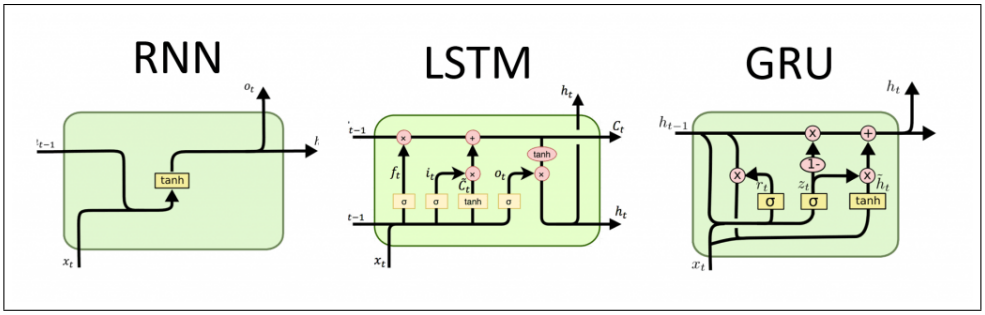

Figure 3.7: RNN, LSTM and GRU taken from [32].

network *memory* of what it had seen before and allows it to model a next value in a sequence. RNNs suffer from the vanishing (or exploding) gradient problem where values could become so minimalized as to be insignificant or so maximized that they overwhelm the rest of the network. LSTMs were designed to address the vanishing gradient problem by introducing specific memory cells in the hidden layers which allow cells to *forget* information. LSTMs are able to learn very complex patterns in NLP.

GRUs extend LSTMs with an update gate to govern the amount information that is passed, or not passed forward. LSTMs and GRUs work in one direction, left to right, or sequentially. Bi-LSTMs, on the other hand, work from both direction, left to right, and right to left, when inferring the next value. Transformers remove the sequential requirement entirely and can fill in values between words. Figure 3.7 shows the three architectures.

With the release of the Transformer paper, *Attention is all you need*[112], attention and multi-head training became a very successful methodology which overcomes

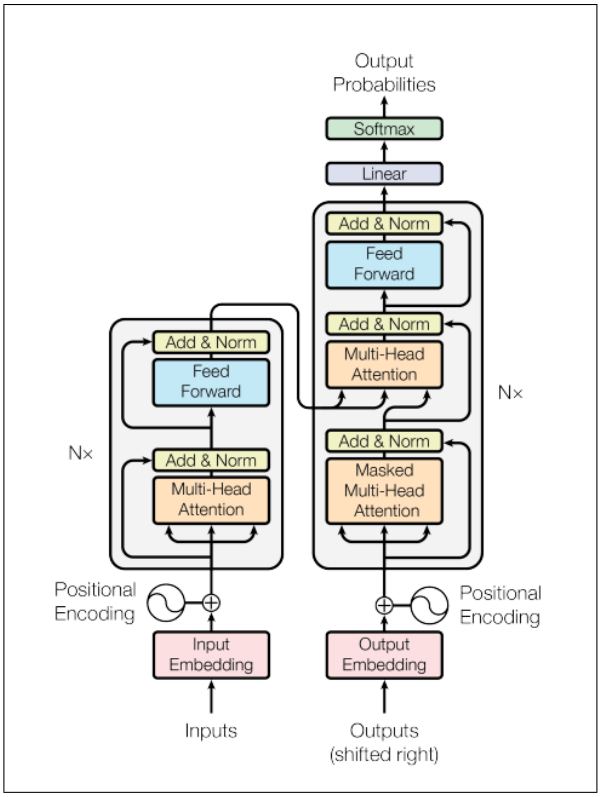

Figure 3.8: Transformer model architecture taken from [112].

some of the limitations of of RNNs. Figure 3.8 shows the basic transformer model architecture.

3.2.3 NLP Metrics

In order to be able to measure the quality of the output of many of these tasks, such as caption generation, it is necessary to have some metrics. There are several choose from among NLP literature: BLEU[82] is one of the oldest and most used

but has some limitations. A couple more recent metrics, SPICE[15] and CIDEr[114], try to overcome those limitations and are gaining popularity. Smoothed-BLEU[21], WMD[62], ROUGUE[65, 35], TER[105], Meteor-1.5[30] and Skip Thought[55] are also options for sentence evaluation. Some even use a composite of these metrics for optimization[96] but all have one limitation or another. Let's discuss BLEU first and it will further our discussion of the others.

BLEU is a form of precision, in the machine learning sense, but modified. In machine learning, precision is defined as the number of true-positives over the number of true-positives plus false-positives, $P = \frac{tp}{tp+fp}$. Adapting an example from Wikipedia[8], precision in NLP is given by, $P = \frac{m}{w_t}$, where m is the number of words in the candidate sentence that appear in the reverence sentence and w_t is the total number of words in the reference sentence. However, precision alone, used in this way is not always informative and can give a misleading result. For example, given the reference sentence, *the ball is on the table*, and the candidate sentence, *the the the the the the*, the precision would be, $P = \frac{6}{6} = 1$. BLEU modifies this by taking the maximum count for each word in the reference sentence and clips the word count in the candidate sentence to this max. The modified precision, or BLEU score, would then be, $P = \frac{2}{6} = \frac{1}{3}$. In practice, a single word is not sufficient for comparisons so n-grams are used which are "a continuous sequence of n items from a given sample of text or speech"[10]. It was found that n-grams of length 4, is the most consistent with human understanding, hence BLEU-4, but n-grams of 3 and 5

also give comparable results[82].

BLEU does not *understand* the meaning of the words or even consider sentence structure. It does correlate relatively well with human understanding but it is far from perfect. The other metrics try to overcome the limitations in BLEU but BLEU is still widely used. A good article about the shortcomings of BLEU, and a run-down of alternative metrics, suggests using caution when optimizing for any of these metrics because they all have some limitation depending on the task[109]. That author suggest even more alternatives to BLEU, not listed above, such as Word Error Rate (WER)[13, 56], Perplexity[11], F-Score[9], STM[69], NIST[66], TERp[106], hLEPOR[41], and RIBES[47].

CIDEr, by contrast, uses term-frequency inverse document-frequency (TF-IDF) to aggregate n-grams across the entire dataset. The motivation is that words which are present in all captions carry less information and so are weighted accordingly. Many of the common NLP metrics use n-grams and can generate false positives when there is a lot of overlap of n-grams between images. SPICE addresses this taking the F-score over tuples of candidate n-grams.

3.2.4 Joint-Embeddings and Cosine Similarity

For our purposes, we are mainly focused on optimizing BLEU as a guide to whether our captions are improving or not. We quickly saw the need for a more objective metric that can be used when there is no reference sentence to compare a candidate

with. This led us to another search for a metric and found several studies that use *cosine similarity* as a way to measure quality of a caption, given an image and sentence pair[74, 89, 79, 98, 63].

Using cosine similarity, a large model may be trained on a joint, visual-linguistic embedding space and distances within that space can be measured. The vectors for the shared embedding space can be created by concatenation of the image and sentence vectors, addition, or multiplication. Other very similar and notable approaches use Bi-Linear Pooling and Fast Fourier Transforms[25] and joint embeddings with video vectors and sentences[64].

To use cosine similarity in a retrieval model for a joint image and language space, we follow [33], [74], and [115].

Given an image I and caption c, use domain specific feature encoders, $\phi(I)$ and $\psi(c)$, and project the features into a joint embedded space by W_I and W_c.

$$f(i) = W_I^T \phi(I) \tag{3.1}$$

$$g(c) = W_c^T \psi(c) \tag{3.2}$$

The similarity between them is the cosine similarity or euclidean distance taken by the dot product in the embedding space, normalized over the unit hypersphere.

$$s(I, c) = \frac{f(I) \cdot g(c)}{\|f(I)\| \|g(c)\|} \tag{3.3}$$

The value of s ranges from -1 to 1, where closer to 1 means the image and caption are more similar and closer to -1 means that the image and caption are less similar. Pretrained network models may be used in this application and the only thing that needs to be learned are the transformations for W_I and W_c which can be done with two non-linearities, ReLU and L2 norm.

In Figure 3.9, taken from [115], the center portion, *Embedding Network*, shows the two paths through the architecture for images and captions. W_1 and W_2, and V_1 and V_2 are non-linear layers in the branches. Embedding Loss is the cosine similarity measure.

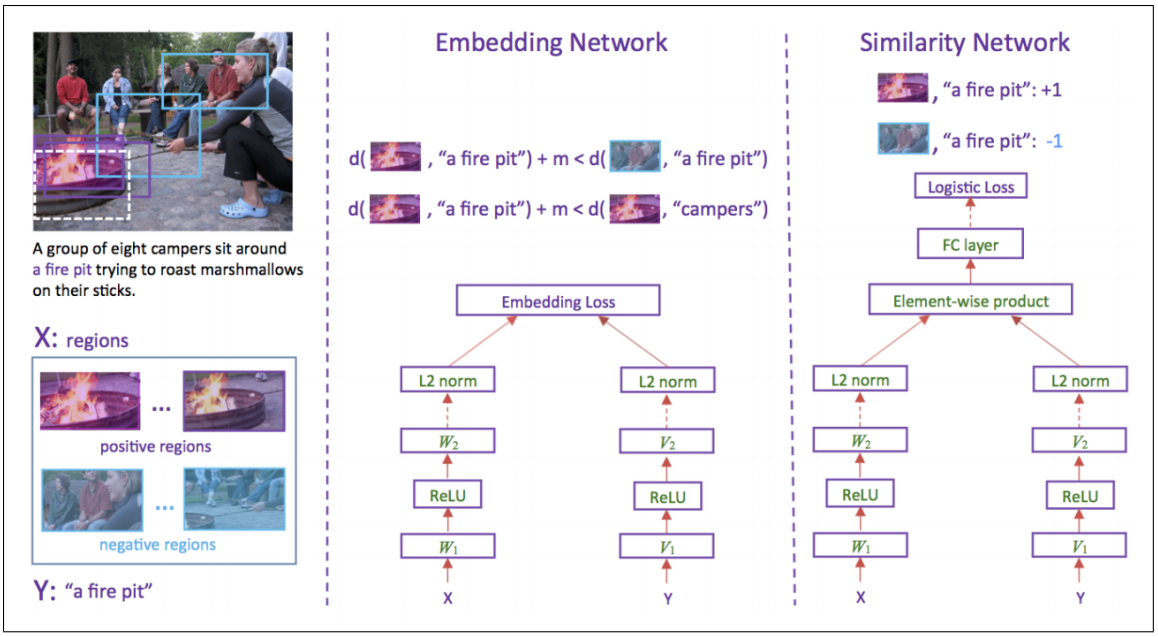

Figure 3.9: Figure taken from [115].

The authors of [115] also propose a new architecture, on the right, the *Similar-*

ity Network, which takes the dot product of the output feature vectors from both branches, computes the dot product and then runs them through a full-connected, non-linear layer, and logistic loss function. They say this alternative architecture is simpler but less flexible.

Another noteworthy architecture, SC-RANK, shown in Figure 3.10 from [119], uses cosine similarity and a ranking metric for Reinforcement Learning (RL). They call this Self-Critical Ranking, hence SC-RANK. RL is another area of deep learning where a reward is given for correct answers and fed back to earlier layers in the network for feedback while training.

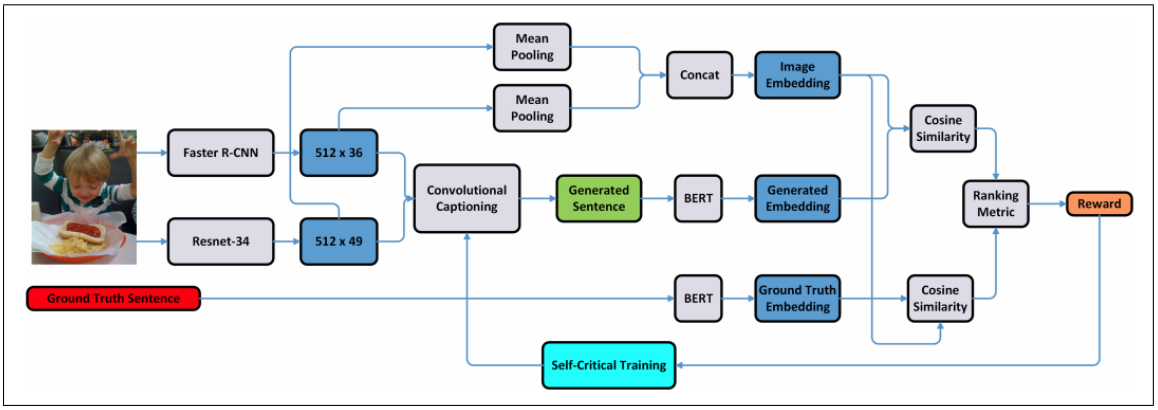

Figure 3.10: SC-RANK network architectures uses cosine similarity and a ranking metric for RL[119].

3.3 Multi-Modal Learning

Current state of the art models for computer vision and NLP are use feature vectors from multiple architecture modes. Joint-embeddings are a type of this multi-modal approach. Other architectures include 12-in-1[72], VilBERT[71], Policy Gradient[70], GAN[26], GLACNet[54, 107], Pythia[49], SC-RANK[119], VSEPP[33], BERT[31], and Transformer[112].

We've chosen to work with the Pythia for our study because it is not only a model but also a "modular framework for vision and language multi-modal research"[1]. Pythia is planned to be replaced by MMF, multi-model framework, and which is currently in beta[102].

We've also chosen to work with VSE++[33], Visual Semantic Embedding Plus Plus, which is another multi-modal framework, that allows us to calculate cosine similarity for image and caption pair against a pretrained COCO model.

Additionally, there are several promising new captioning projects that leverage multi-modal learning for vision and language. We did not investigate them thoroughly here but they should definitely be considered in future work. These include, [80], [75], and [73].

[1] https://github.com/facebookresearch/mmf

Chapter 4

Datasets

In order to experiment with transfer learning, we searched for datasets that are relevant to our study. We found many good candidate datasets that fall into several broad categories: Object Datasets, Cooking Datasets, Nutrition and Food Items, Visual QA and Visual Description, and Movie Datasets.

Object Datasets

The smaller, common object detection, segmentation, and classification datasets, such as PASCAL[85], Flikr8k[44], and Flikr30k[121] have thousands or tens of thousands of image. They are good for toy problems and developing algorithms. The larger datasets such as MS COCO[67], ImageNet[29], ObjectNet[19], and Conceptual Captions[97] are good for transfer learning pretrained models to new domains. These object datasets are open domain, open orientation, and contain a good general sampling of real-world objects which can be used as a starting point in fine-tuning

models for other domains.

Cooking Datasets

Since we are interested in transfer learning from a general domain to cooking, we also searched for relevant cooking datasets. Among those, there are the MPII Cooking Dataset [93], YouCook[28], YouCookII[124, 123], TACoS[87], TACoS Multilevel[90], Recipe 1 Million[94], Recipe 1 Million+[76], and RecipeQA[118]. Recipe 1 Million and Recipe 1 Million+ look to be promising datasets to use in the future as more computational resources become available.

We chose to work with the YouCookII dataset first because it was a medium sized dataset, which would provide enough samples to demonstrate effectiveness and was still small enough to fit within the hardware and compute constraints, and complete training in a short enough time. The assumption was that there were several good options but starting with one allowed us to learn how to integrate a dataset into the Pythia framework which would be knowledge we can extend to other datasets in the future.

The YouCookII dataset is a publicly available dataset of cooking videos, created by graduate students at the Michigan State University[1]. It is comprised of 2000 long, untrimmed, YouTube videos with open orientation, most are of a common kitchen scene. The average video length is 5 minutes and 26 seconds. There are 89 recipes, with 22 videos for each recipe. The videos are annotated by humans and broken

[1]http://youcook2.eecs.umich.edu/

down into 15,400 segments or clips. Each segment has one description. There are 1333 training videos, 457 validation, and 210 testing. Segment annotations are only provided for the training and validation sets. The annotations are in the form of imperative instructions. Figure 4.1 shows how a video is broken into segments and annotated.

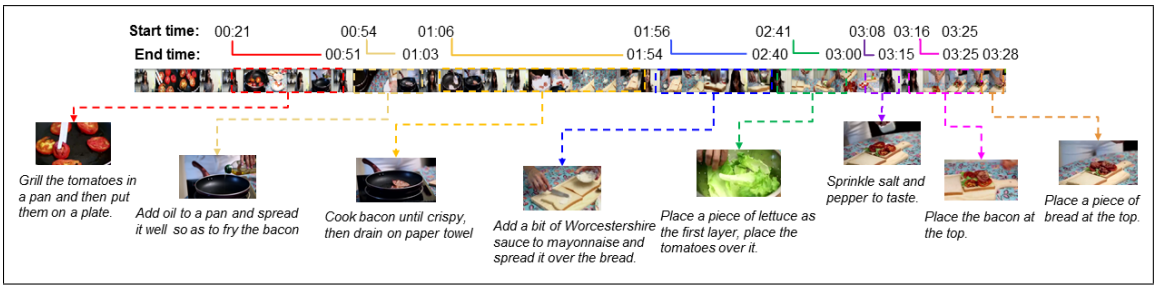

Figure 4.1: How a video is broken into segments and annotated in YouCookII[124].

Nutrition and Food Item Datasets

Since we are interested in cooking, there are several food-based and nutrition datasets that may be useful: Food-101[20], PFID[23], Food State ID[48], VegFru[45], Grocery Store[57], NutriNet[77], DeepFood[81, 68], and Foodcam[51].

Visual QA and Visual Description Datasets

At some point, in the YouDescribe study, we will want to be able to answer real-time questions from blind and visually impaired users, about the videos they are watching. There are several good Visual QA and Visual Description datasets that could

be used for that: CoQA[86], Visual Storytelling[46], Kinetics[52], UFC101[108], HMDB[61], VQA[17], VizWiz[40], TextVQA[104], Visual Dialog[27], CLEVR[50], CLEVR-Dialog[58], YouTube2Text[39], and VideoStory[36].

Movie Description Datasets

There are many movie description databases which can be used for when we extend our image-based captioning work toward video-based descriptions. These include: MSVD[22], MPII Movie Description[92], M-VAD[91], MSR-VTT[117], VATEX[116], Charades[99, 100], VTW[122], and ActivityNet Captions[59]].

Chapter 5

Our Contributions

To be able to run the transfer learning experiments with the YouCookII dataset in the Pythia framework, we needed to preprocess the videos for integration and code the necessary classes into the Pythia Python library. We also built a dataset explorer to get a feel for the data. And we made several Web services which can be accessed either through a Web page or a REST API, to handle captioning and cosine similarity scoring. The Web services were written in Python with the `web.py`[1] library and are running on Amazon Web Services (AWS), Elastic Compute Cloud (EC2) instances.

[1] https://webpy.org/

5.1 VSE++ Cosine Similarity Score as a Service

The VSE++ cosine similarity score Web service uses the code from the `vsepp` project[2] and wraps it in a `web.py` Python Web application. The application exposes a Web HTML form where you can submit an image URL and caption and a REST API that can take an image and caption. Both return a cosine similarity score within the feature space. VSE++ comes with pretrained models, trained on the COCO dataset. Their best model was a ResNet-152, so we used that as the backbone in our app. Their model trains the ResNet with both image vectors and word vectors from the captions in the training set. When the Web application starts, it loads this model into memory. When the application receives a POST request with an image and caption, it retrieves the cosine similarity or distance between the image and caption within a shared visual-semantic embedding space.

Figure 5.1 shows some example images and ground truth (GT) captions from [33] and the Flickr30K dataset[121]. In [33], they are interested in hard-negative (HN) mining. The values next to HN are the cosine similarity values for those hard-negatives shifted to the range $[0, 2]$, as opposed to $(-1, 1)$. The idea is still the same: a ResNet-152 was trained on that image and the GT caption and later a cosine similarity score can be retrieved for that image and another caption or a different image and different caption. We leverage this ability as a service to provide feedback in multiple areas of this study.

[2]https://github.com/fartashf/vsepp

GT: A little girl wearing pink pants, pink and white tennis shoes and a white shirt with a little girl on it puts her face in a blue Talking Tube.

HN: [0.26] Blond boy jumping onto deck.

GT: A teal-haired woman in a very short black dress, pantyhose, and boots standing with right arm raised and left hand obstructing her mouth in microphone-singing fashion is standing.

HN: [0.08] Two dancers in azure appear to be performing in an alleyway.

GT: Two men, one in a dark blue button-down and the other in a light blue tee, are chatting as they walk by a small restaurant.

HN: [0.41] Two men with guitars strapped to their back stand on the street corner with two other people behind them.

GT: A man wearing a black jacket and gray slacks, stands on the sidewalk holding a sheet with something printed on it in his hand.

HN: [0.26] Two men with guitars strapped to their back stand on the street corner with two other people behind them.

GT: There is a wall of a building with several different colors painted on it and in the distance one person sitting down and another walking.

HN: [0.06] A woman with luggage walks along a street in front of a large advertisement.

GT: A man is laying on a girl's lap, she is looking at him, she also has her hand on her notebook computer.

HN: [0.18] A woman sits on a carpeted floor with a baby.

GT: A young blond girl in a pink sweater, blue skirt, and brown boots is jumping over a puddle on a cloudy day.

HN: [0.51] An Indian woman is sitting on the ground, amongst drawings, rocks and shrubbery.

GT: One man dressed in black is stretching his leg up in the air, behind him is a massive cruise ship in the water.

HN: [0.24] A topless man straps surfboards on top of his blue car.

Figure 5.1: Examples from VSE++[33].

The VSE++ Web service runs on p2.xlarge EC2 instance with 1 GPU with 12 gigabytes of GPU memory, and 4 vCPUs with 61 gigabytes of system memory. Even though training may have happened on many GPUs, using 1 is sufficient for

inference. The time between doing inference on 1 GPU vs the CPU is about a 10 times speedup and worth it to run as a service for large batches of queries.

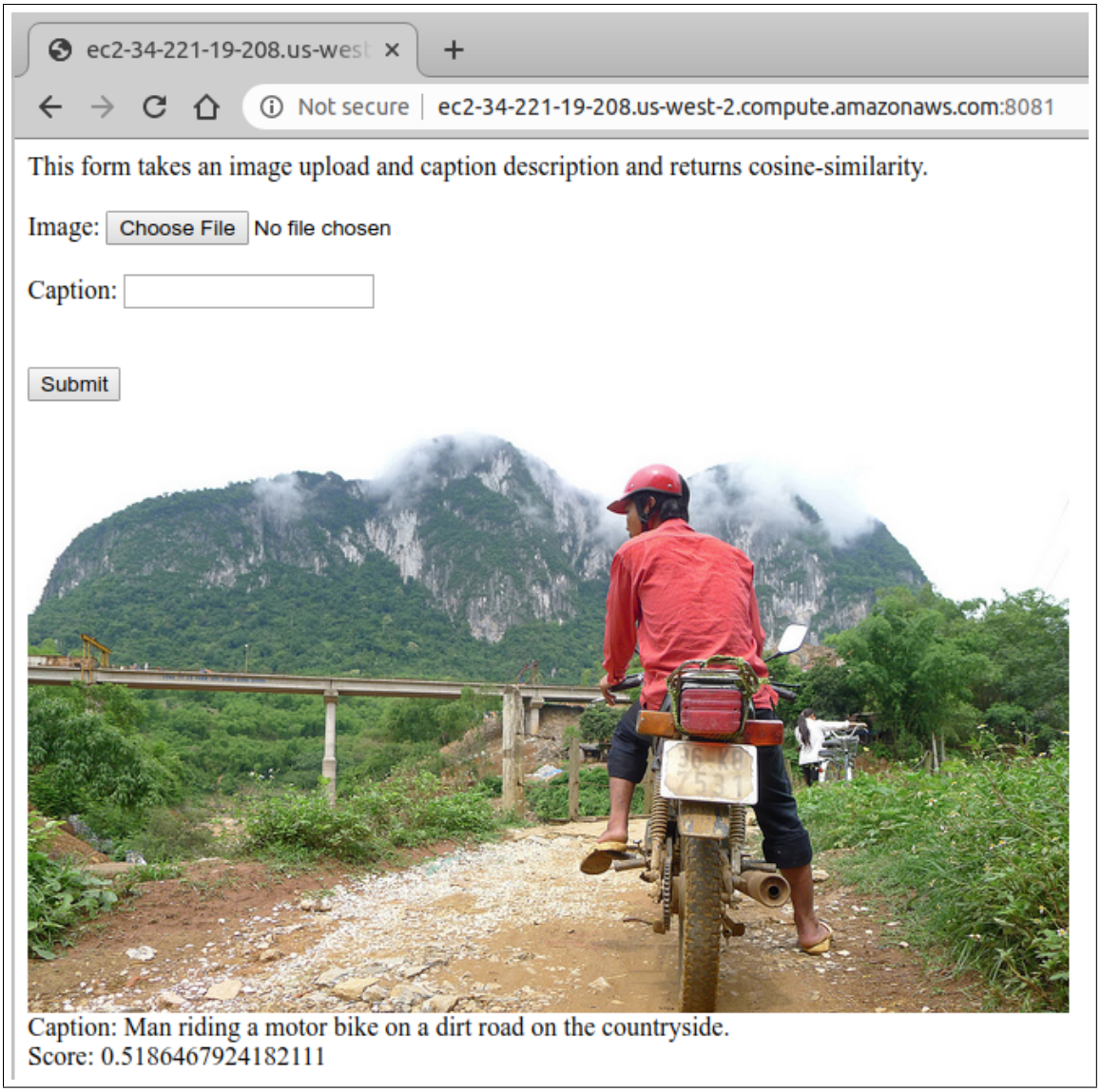

Figure 5.2: Screenshot of Web HTML for VSE++ Web Service.

Figure 5.2 shows a screenshot of the Web form after uploading a picture with a caption. The cosine similarity score is at the bottom for the given image and caption. Code for the site is available at the author's github[3]. The form requires a username and password to access it. The URL for the Web HTML form front end is:

`http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8081/`

The service also provides an `/api` interface and an `/upload` interface. The `/api` interface allows for a `POST` request with image URL and a caption, and the `/upload` interface allows for uploading an image from the local file system and a caption. These require a pre-determined token to access. The URLs for these are:

`http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8081/api`

`http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8081/upload`

[3]https://github.com/ascott02/vsepp/tree/master/webservice

5.1.1 Example 1: Post Image URL and Caption to Web Service

Here is a simple Python command line driver to test the `/api` service. It requires the requests library and a token.

```
import requests
import os

img_url = "https://github.com/ascott02/vsepp/raw/master/
   COCO_val2014_000000391895.jpg"
caption = "Man riding a motor bike on a dirt road on the countryside."

page = 'http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8081/api'
token = ''
def send_data_to_server():
    multipart_form_data = {
        'token': ('', str(token)),
        'caption': ('', str(caption)),
        'img_url': ('', str(img_url)),
    }
    response = requests.post(page, files=multipart_form_data)
    print(response.status_code, response.text)

send_data_to_server()
```

Output:

```
python post_url_to_webservice.py
(200, u'0.5186467924182111')
```

5.1.2 Example 2: Post Image File and Caption to Web Service

Here is a simple Python command line driver to test the `/upload` service. It requires the requests library and a token.

```
import requests
import os

filename = "./COCO_val2014_000000391895.jpg"
caption = "Man riding a motor bike on a dirt road on the countryside."
page = 'http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8081/
    upload'
token = ''

def send_data_to_server():
    multipart_form_data = {
        'token': ('', str(token)),
        'caption': ('', str(caption)),
        'img_file': (os.path.basename(filename), open(filename, 'rb')),
    }

    response = requests.post(page, files=multipart_form_data)
    print(response.status_code, response.text)

send_data_to_server()
```

Output:

```
python post_image_to_webservice.py
(200, u'0.5186467924182111')
```

5.2 Pythia Caption Generation as a Service

The Pythia caption generator Web service is setup in much the same way as the VSE++ service. The Pythia caption generator runs on the same EC2, p2.xlarge, single-GPU instance, is written in `web.py` and loads a BuTD detectron model into memory then listens for POST requests with an image URL and when receives them, generates and returns a caption. The code is available at the author's github[4]. This service provides a Web HTML form front-end where a user can submit an image URL, and a REST API that takes an image URL. The Web form requires a username and password. The API requires a token. The URL for the Web HTML form is:

`http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8080/`

Figure 5.3 shows a screenshot of the Web HTML form after submitting an image and the Web service returning a newly generated caption for the image. Note that this image was not used during training.

[4]https://github.com/ascott02/pythia/tree/master/webservice

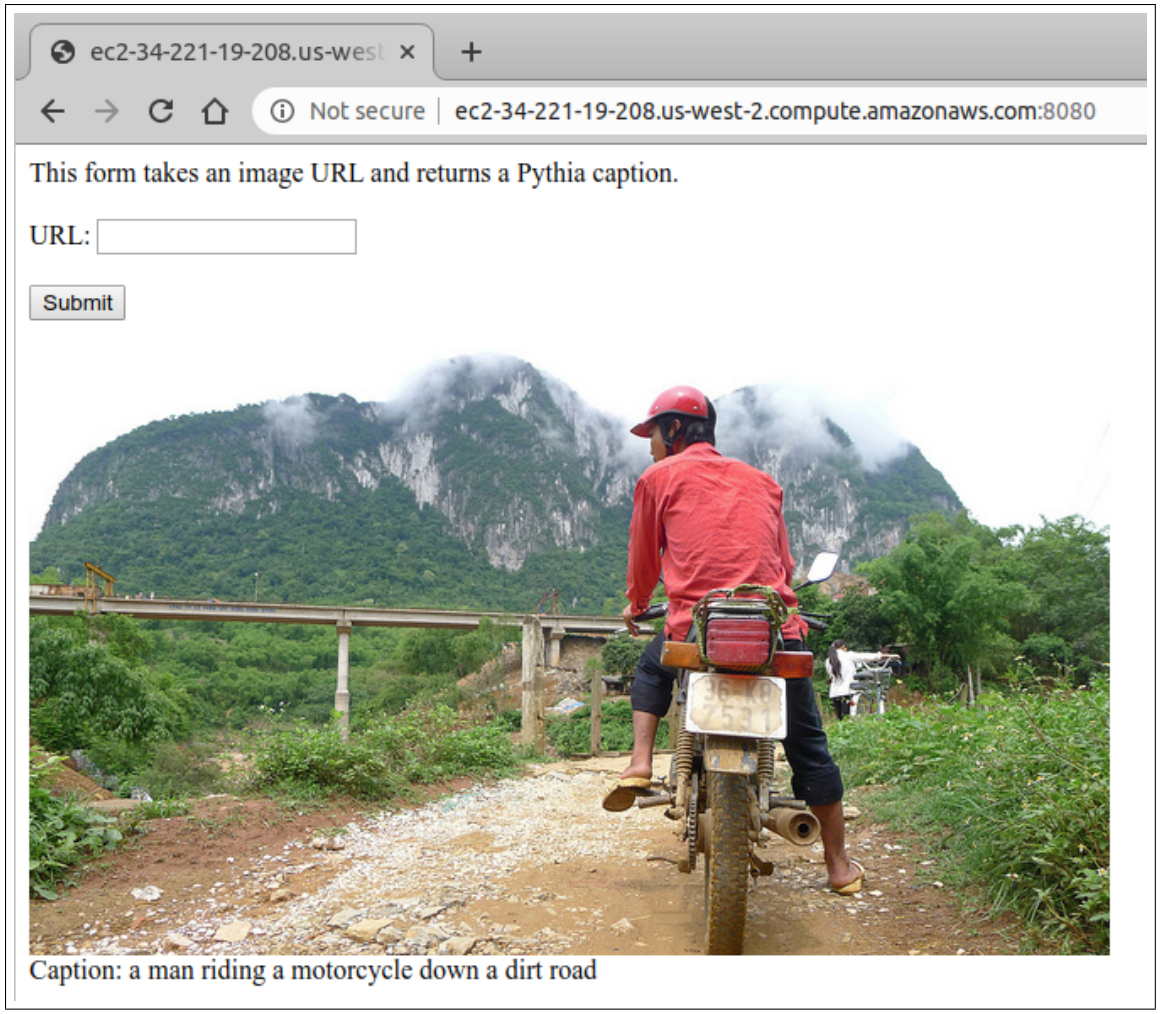

Figure 5.3: Screenshot of Web HTML for the Pythia Captioning Web Service.

There is also an Easter Egg Meme Generator that will overlay the generated caption with Impact font. Figure 5.4 shows a screenshot of the meme generator. The URL for that is:

`http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8080/meme`

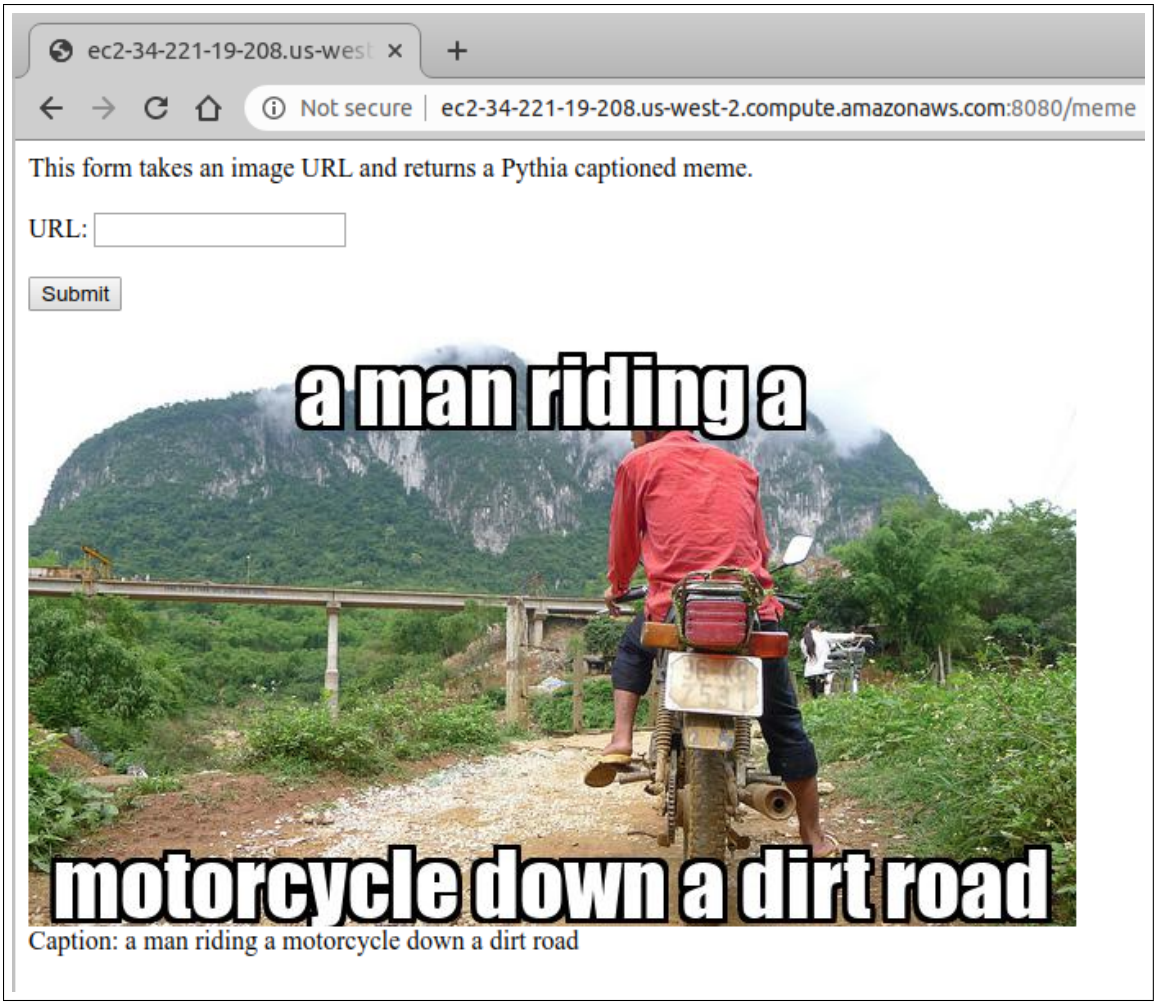

Figure 5.4: Screenshot of Pythia Captioning Meme Generator (overlays caption with impact font).

5.2.1 Example 3: Post Image URL to Pythia Captioning Web Service

The URL for the API is:

```
http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8080/api
```

Here is a simple Python command line driver to test the `/api` service. It requires the requests library and a token.

```
cat post_url_to_service.py
import requests
import os

image_url = "https://github.com/ascott02/vsepp/raw/master/
    COCO_val2014_000000391895.jpg"
page = "http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8080/api"
token = ''

def send_data_to_server():
    multipart_form_data = {
        'token': ('', str(token)),
        'image_url': ('', str(image_url)),
    }
    data = {"token": str(token), "image_url": str(image_url)}

    response = requests.post(page, data)
    print(response.status_code, response.text)

send_data_to_server()
```

Output:

```
python post_url_to_service.py
(200, u'a man riding a motorcycle down a dirt road')
```

Now that we have these Web services, we will leverage them for data preparation and experiments.

5.3 YouCookII Data Preparation

As a side note, obtaining the data was not trivial. The authors provide the dataset as ResNet-34 precomputed feature but these were not compatible with Pythia. We had to source all of the videos from the YouTube links in the annotations file. Many of the YouTube videos were no longer available. After getting the bulk of them downloaded, we were able to contact the authors of the dataset for the missing videos. Once we had a complete dataset, we prepared the data for integration with Pythia.

Preparing the data involved extracting keyframes from the segments, extracting features for the keyframes, and building a vocabulary from the captions. To extract keyframes from the segments, we extracted 1 frame per second, from each segment. Then, we used the VSE++ Web service to score each frame with the caption for that segment. We kept the frame that has the highest cosine similarity with the caption for that segment. We repeated this for every segment in every video in the training and validation set. We dropped any keyframes and captions with a cosine similarity score less than .17. Figure 5.5 shows the extraction process.

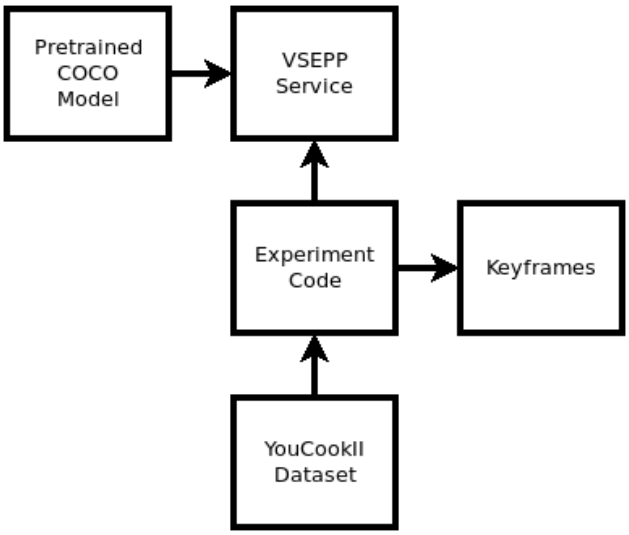

Figure 5.5: Extracting keyframes from YouCookII dataset.

After extracting the keyframes, we then shuffled all of the keyframe and caption pairs, and split them into three new subsets: training, validation, and test at percentages of 66, 22, and 12, respectively. This resulted in 7391 training samples, 2463 validation samples, and 1345 test samples. Figure 5.6 shows some of resulting keyframe images and their corresponding ground-truth captions from the test set that were extracted from the YouCookII videos.

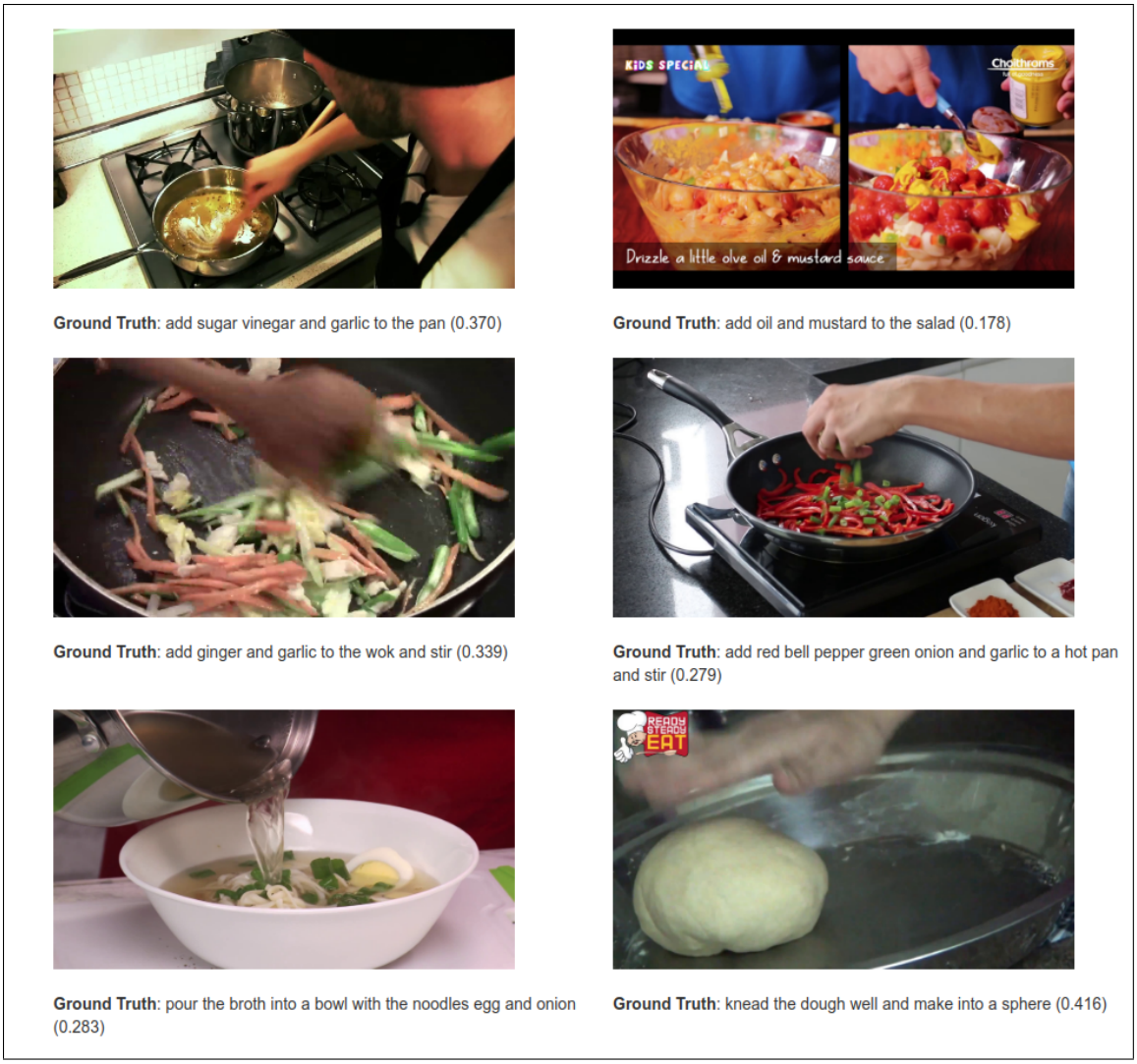

Figure 5.6: Example test set images and captions created from the YouCookII dataset.

5.4 YouCookII Dataset Explorer

To get a better feel for the YooCookII videos, annotation segments, and extracted keyframes we created a YouCookII Dataset Explorer, using `web.py` and running on an EC2, non-GPU instance. The code for this tool is available at the author's github[5]. This application simply takes the annotations file, and makes a Web form with a pull-down input that has each of the videos. When a video is selected and form is submitted, the video is loaded in an embedded HTML frame and is shown along with the video URL, the video ID, which subset the video originally came from, the duration, the type of recipe, and the segment times and captions for each segment. A screenshot of the application can be seen in Figure 5.7.

Below the video and annotation, are also each individual keyframe shown with the caption the cosine similarity for that keyframe and caption. Figure 5.8 shows a couple of the keyframes, captions, and scores for the video in Figure 5.7. The explorer was useful to us to see how the automatically extracted keyframes based on cosine-similarity value lined-up with the video and description for the segment.

[5]https://github.com/ascott02/youcookIIexplorer

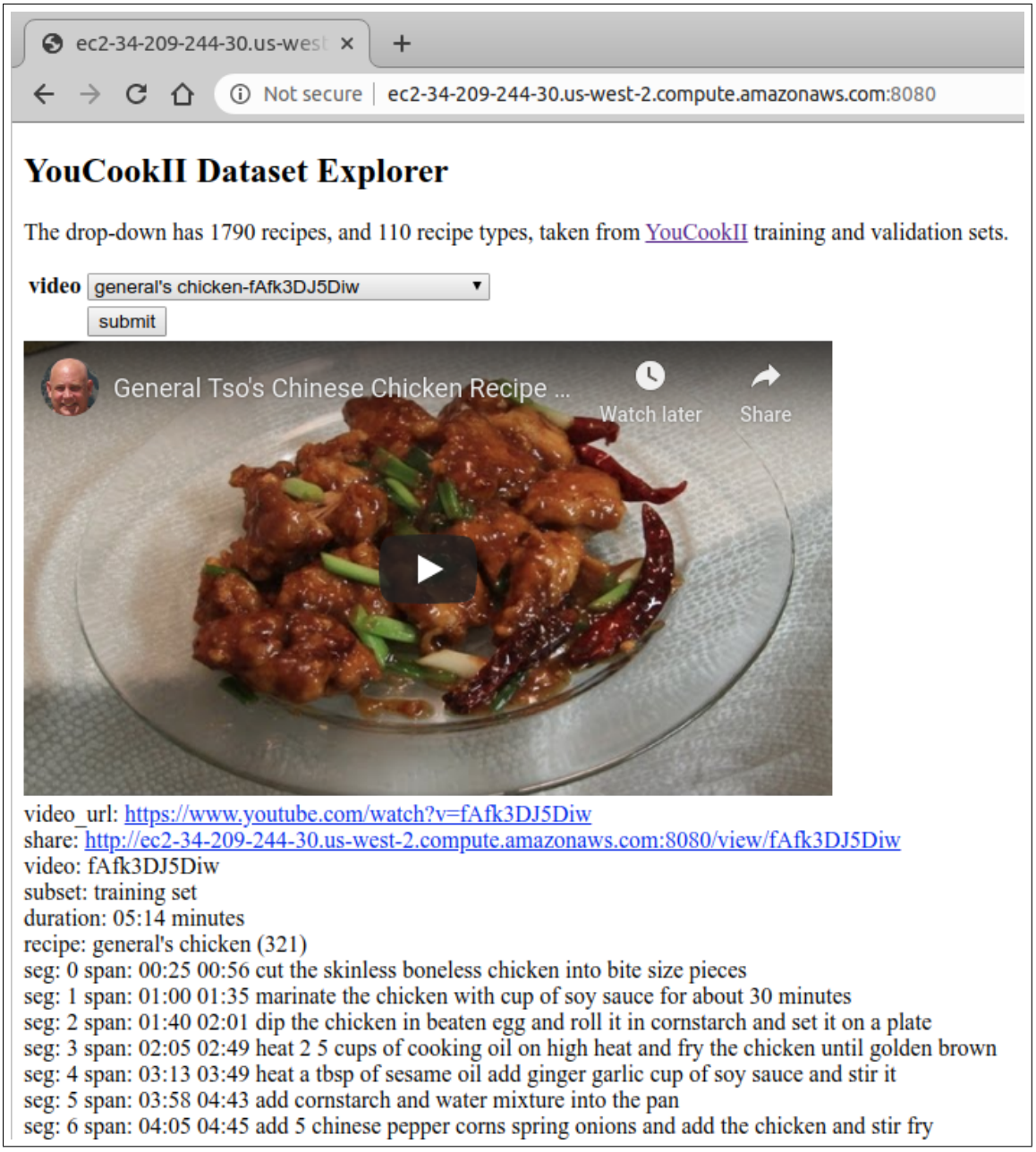

Figure 5.7: Screenshot of YouCookII dataset Explorer.

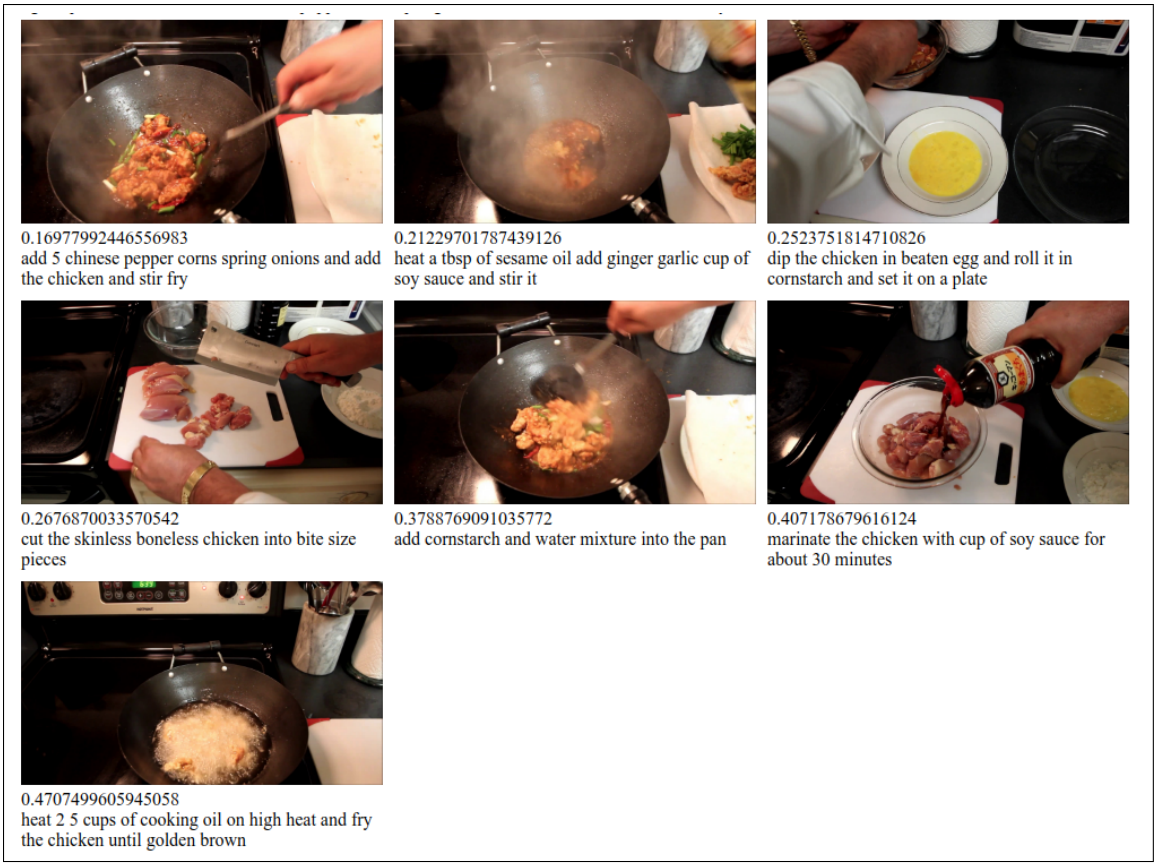

Figure 5.8: Screenshot of YouCookII dataset Explorer keyframes, scores, and captions.

5.5 YouCookII Integration with Pythia

Integration into Pythia was fairly straight-forward after overcoming the learning curve. Pythia does not handle the data directly, but rather expects that features to be already extracted and works with `.npy` files. Scripts in the library were repurposed for extracting ResNet-152 and Detectron features. Then, following the tutorial on the Pythia documentation site[6], we created the necessary classes and configuration files for the YouCookII dataset. This involved creating the Builder class for YouCookII, two configuration files pointing to the correct resources, a Dataset Class and metric where we reused the COCO BLEU4 metric as the YouCookII BLEU4 metric. We leveraged examples from other tasks to help with integration. We also extracted a vocabulary from the YouCookII segment annotations to be used later when expanding the vocabulary for COCO training. All of these files can be found in the author's github[6]. In addition, there are other utility scripts, in that directory, for building the dataset splits, creating IMDB (image database) files, and extracting the vocabulary from the YouCookII dataset annotations.

[6]https://github.com/ascott02/pythia/tree/master/pythia/tasks/captioning/youcookII

Chapter 6

Hypothesis and Design

The central hypothesis of this work is that image caption quality may be improved for a specific domain by fine-tuning a pretrained model on a domain-specific dataset. In other words, image caption quality may be improved by using transfer learning.

To experiment with this we needed the VSE++ cosine similarity Web service, the Pythia captioning service, and YouCookII dataset integration with Pythia, as outlined above. For each experiment, we followed this general procedure:

- Train a Pythia model or use a pretrained one
- Load the model into the Pythia Web service
- Generate captions for images in YouCookII test set
- Score the images and captions using the VSE++ Service

Figure 6.1 illustrates this general procedure.

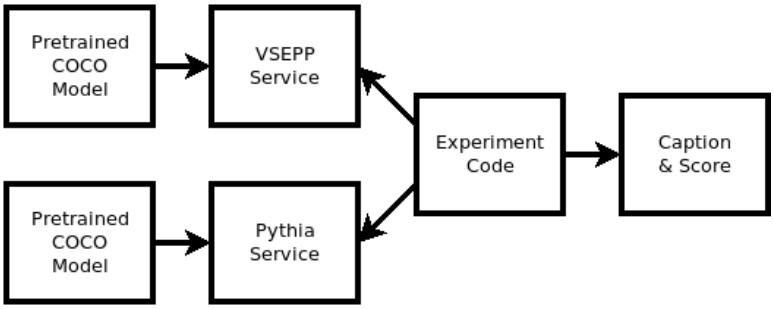

Figure 6.1: Using pretrained models as services to generate captions and scores.

We then perform additionally, fine-tune training on the COCO pretrained model using the YouCookII training and validation sets, then run the procedure again, as shown in Figure 6.2.

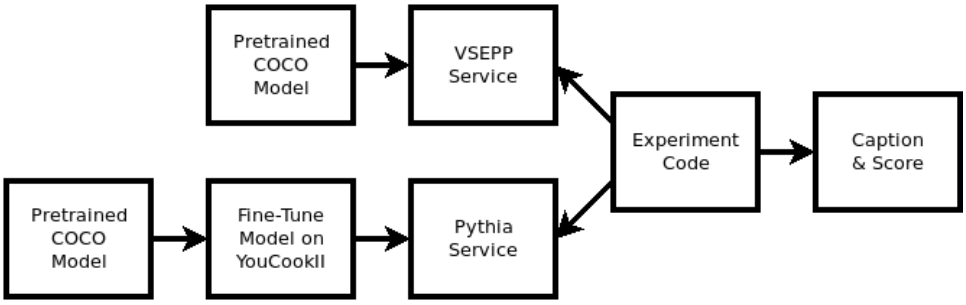

Figure 6.2: Fine-tune training COCO pretrained model with YouCookII dataset.

One of the big questions in using the transfer learning and transfer metric approach was what to do with the different vocabulary. Not all words in the YouCookII vocabulary were present in the COCO vocabulary. The VSE++ model was trained on joint-embeddings of the COCO dataset. The Pythia model was also trained

on the COCO dataset and vocabulary. In order to better understand the effect of vocabularies on the models we experimented with and without expanding the vocabulary. We did not have time to retrain VSE++ on an expanded vocabulary but future work should include this step.

We were then able to run several experiments: A baseline, training COCO in Pythia with default COCO vocab; Training COCO in Pythia with an expanded COCO and YouCookII vocabulary combined; Training YouCookII without loading a pretrained model and using the default COCO vocabulary; Training YouCookII without loading a pretrained model but using the YouCookII vocabulary; Fine-tune training a pretrained COCO model with the YouCookII dataset and default COCO vocabulary; and Fine-tune training a pretrained COCO module with the expanded vocabulary.

Chapter 7

Experiments

For each experiment we ran, we used a workstation with 3 × Nvidia GTX 1080 GPUs (8 Gigabytes of GPU memory each), Intel Core i7-9800X CPU @ 3.80GHz CPU (16 cores), with 32 Gigabytes of system memory, and SSD hard drive. The operating system was Ubuntu 16.04 LTS. We used Python virtualenv environments with Python 3.5 and PyTorch 1.2. For each experiment we used the detectron features, glove embeddings, batch size 192, and learning rate of 0.01.

We found that, in general, batch size seems to be best at around 32-64 samples per GPU on a 8 Gigabyte GPU system, and using image size of ($224 \times 224 \times 3$) (color images). This means that with 3 GPUs, and batch size of 192, each GPU could be working on 64 samples in each batch. Training time took approximately 24 hours for each run, up to 50,000 iterations. Pythia does not use epochs by default. It uses iterations which refers to one training iteration.

For baseline comparisons, we generated histograms and boxplots of the cosine

similarity scores from the default YouCookII images and captions, for the training, validation, and test sets. These are shown in Figure 7.1.

Table 7.1 shows the number of samples in each subset, and the values for minimum, maximum, mean and standard deviation. It's interesting to note the mean and standard deviation are the same for each subset which is good because it means we have a very similar population sample even though very different numbers of samples, which we would expect. The minimums are all 0.17 because we dropped samples where the images and captions had cosine similarity scores lower than 0.17.

Subset	#Samples	Min	Max	Mean ± Std
Training	7,391	0.1700	0.6642	0.3295 ± 0.0096
Validation	2,463	0.1702	0.6525	0.3273 ± 0.0097
Test	1,345	0.1705	0.6123	0.3298 ± 0.0096

Table 7.1: Number of samples, minimum, maximum, mean and standard deviation of cosine similarity scores for all three sets, using the default captions provided.

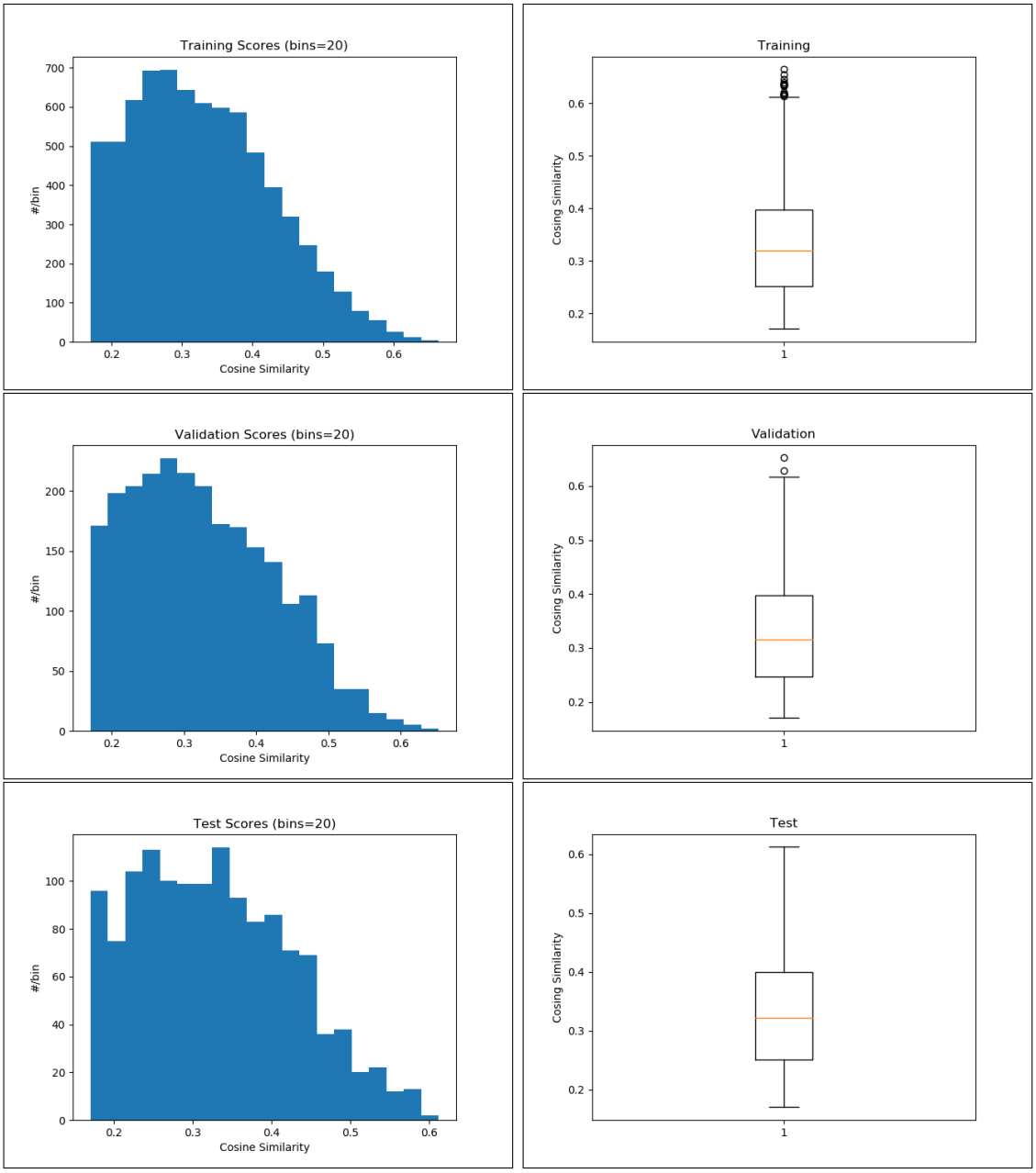

Figure 7.1: Histograms and boxplots for cosine similarity scores of the default YouCookII captions in each set. Keyframes with cosine similarity score less than 0.17 were dropped while creating the dataset.

7.1 Train Pythia on COCO with COCO Vocabulary

For this experiment, we trained COCO from scratch with Pythia, using data parallel, on 3 GPUs, up to 50,000 iterations. For this run, we included the object character recognition (OCR) feature which pulls words out of images. Figure 7.2 shows the tensorboard output of BLEU4 score and the sentence cross-entropy loss during training for the experiment. Figure 7.3 shows the validation scores. The final BLEU4 score on the test set was 31.97 with cross entropy loss of 20.96, for this run. The reported test set value from the Pythia documentation for this run is 35.42. Contributors to the project say that the difference is because of using greedy search versus beam search during inference but we were unable to get inference to run on this model using beam search.

This experiment shows that our fork of the upstream Pythia code is working as expected in our environment. We are now prepared to proceed with the rest of the experiments. Figures 7.2 and 7.3 show the tensorboard training and validation output. The performance of captions and cosine similarity scores are shown in Figure 7.4.

What we see in the tensorboard output is that training converges on the expected BLEU4 score and caption cross-entropy loss is minimized further and further. Similarly, the validation tensorboard output also shows steady convergence on the expected BLEU4 score and cross-entropy loss is also minimized asymptotically, as expected.

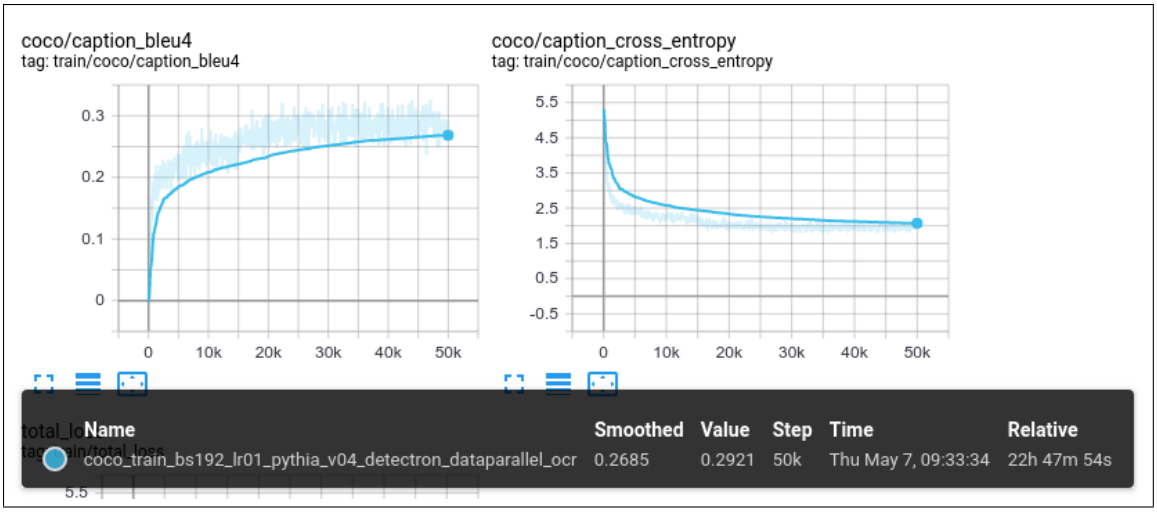

Figure 7.2: Tensorboard visualization of training iterations.

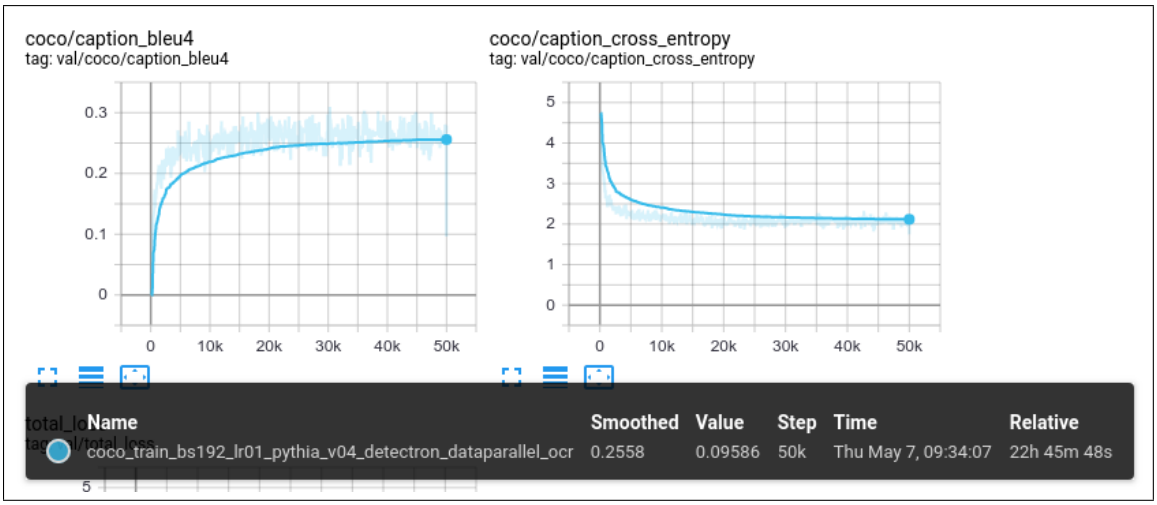

Figure 7.3: Tensorboard visualization of validation iterations.

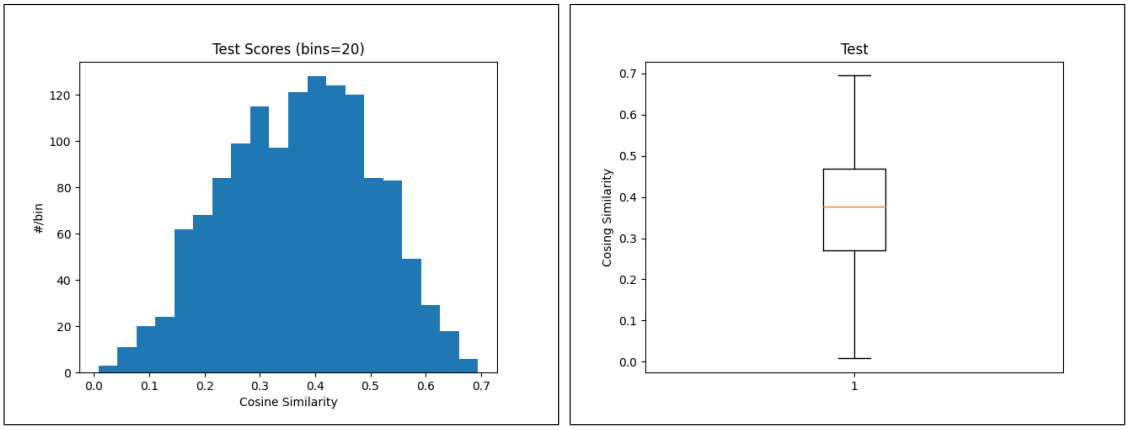

Figure 7.4: Histogram and boxplot of cosine similarity scores from training COCO from scratch with default COCO vocabulary.

The histogram and boxplot show the distribution of cosine-similarity scores on the Pythia generated captions. The Pythia-generated captions have a cosine-similarity distribution and mean that is greater than the ground-truth captions but this is not an exact measure of the quality of those captions. In the Appendix, we show some of the images and captions from all experiments to get a human feel for the overall quality.

7.2 Train Pythia on COCO with COCO and YouCookII Vocabulary

For this experiment, we trained COCO from scratch with Pythia, using data parallel, on 3 GPUs, up to 50,000 iterations, but we additionally added the vocabulary from the YouCookII dataset to the COCO vocab. Figures 7.5 and 7.6 show the training and validation tensorboard output. Figure 7.7 shows the results of this training run. The final BLEU4 score on the test set was 0.3147 and cross-entropy loss was 20.88.

The tensorboard output for training and validation show steady convergence toward reported values, as expected, and just like training with COCO default vocab only. The overall distribution is greater than the ground-truth but we still need a human feel for whether these captions are good or not.

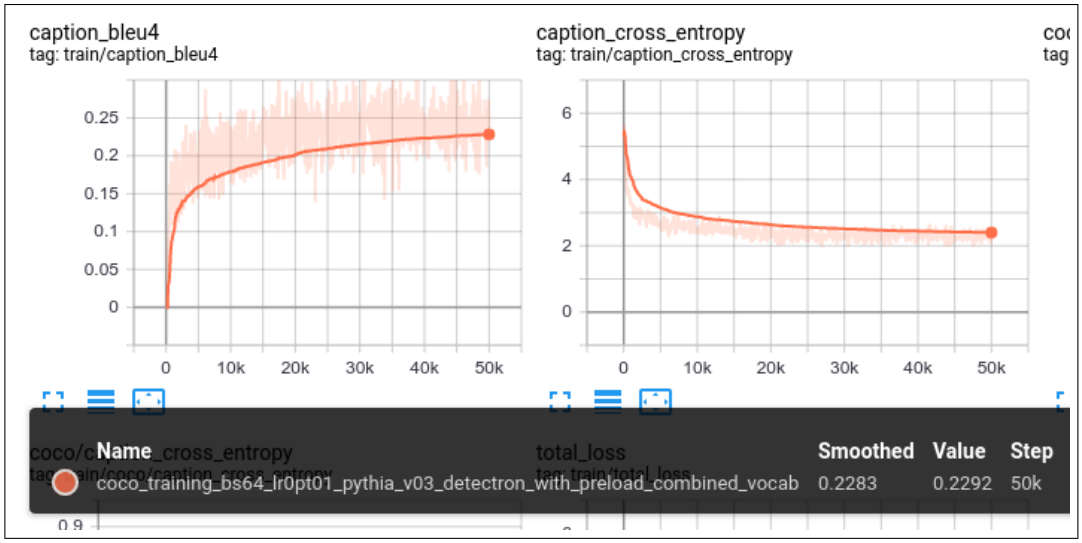

Figure 7.5: Tensorboard visualization of training iterations.

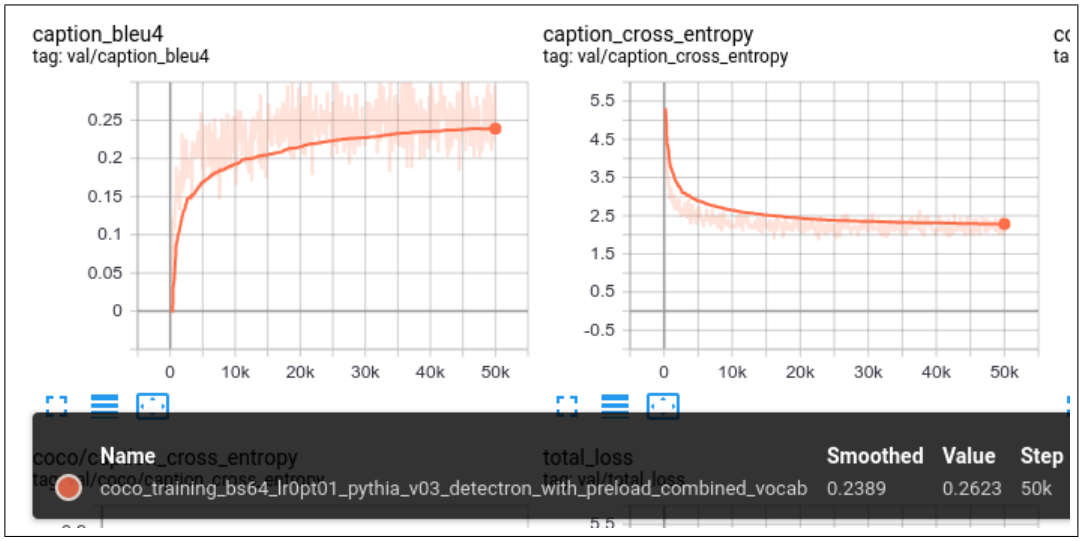

Figure 7.6: Tensorboard visualization of validation iterations.

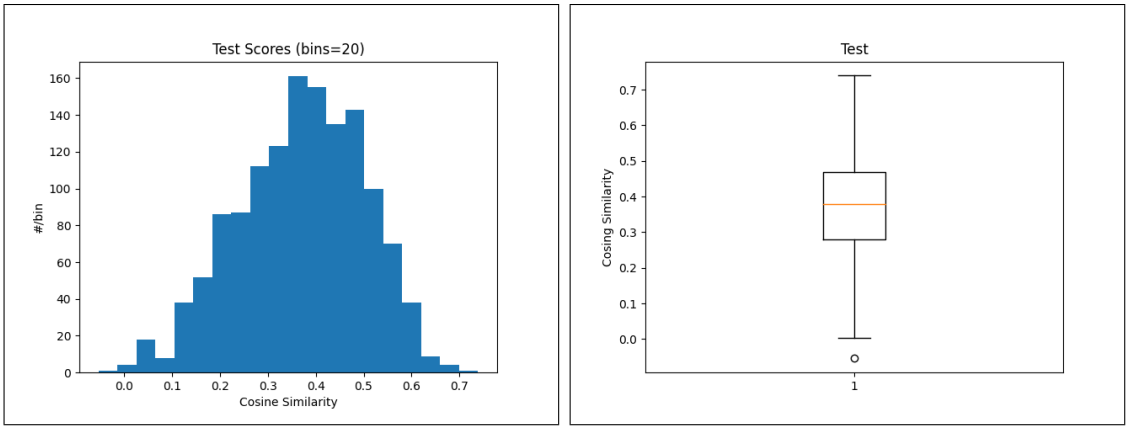

Figure 7.7: Histogram and boxplot of cosine similarity scores from training COCO from scratch with COCO and YouCookII vocabulary.

7.3 Train Pythia on YouCookII without Preloading and with COCO Vocabulary

For this experiment, we trained YouCookII from scratch with Pythia v0.3, using data parallel, on 3 GPUs, up to 50,000 iterations. We did not preload a model trained on COCO, and we used the default COCO vocabulary. Figure 7.8 shows the tensorboard output of BLEU4 score and the sentence cross-entropy loss during training for the experiment. Figure 7.9 shows the validation scores. The final BLEU4 score on the test set was .

The tensorboard output shows steady convergence on BLEU4 of 100.0, which is unrealistic. Cross-entropy is minimized well during training so the network seems to learn something but the validation convergence is very poor and validation cross-entropy does not converge toward a minimum. This means that there is probably either a problem with the test set or our experimental procedure. We are uncertain whether to trust these results. However, the cosine-similarity score distribution, while lower than the baseline and lower than the COCO training, still shows positive results, so we can infer that the network had learned something. We will need to review the actual captions output to see how well this performed. See the Appendix for image and caption samples from all experiments.

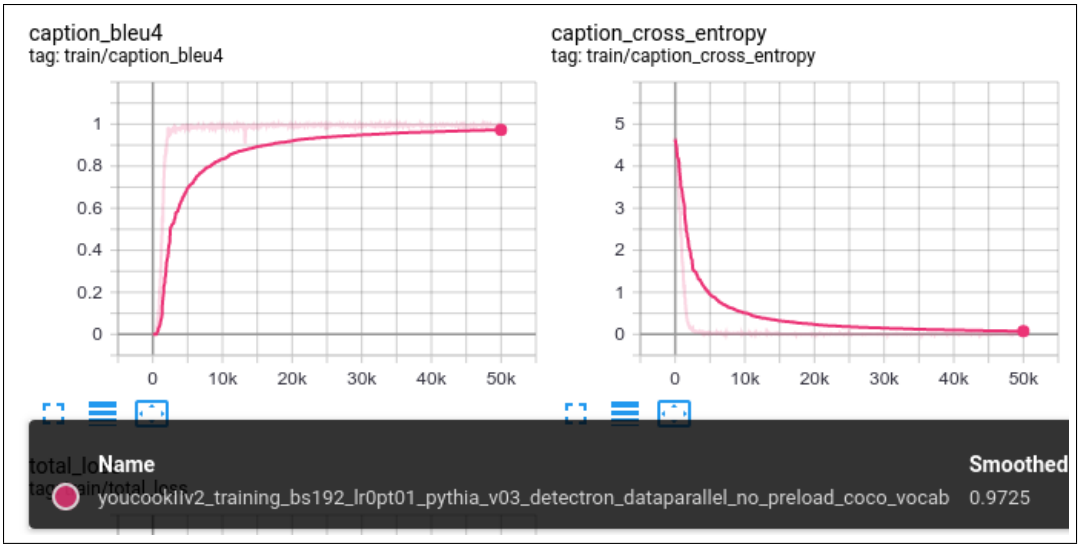

Figure 7.8: Tensorboard visualization of training iterations.

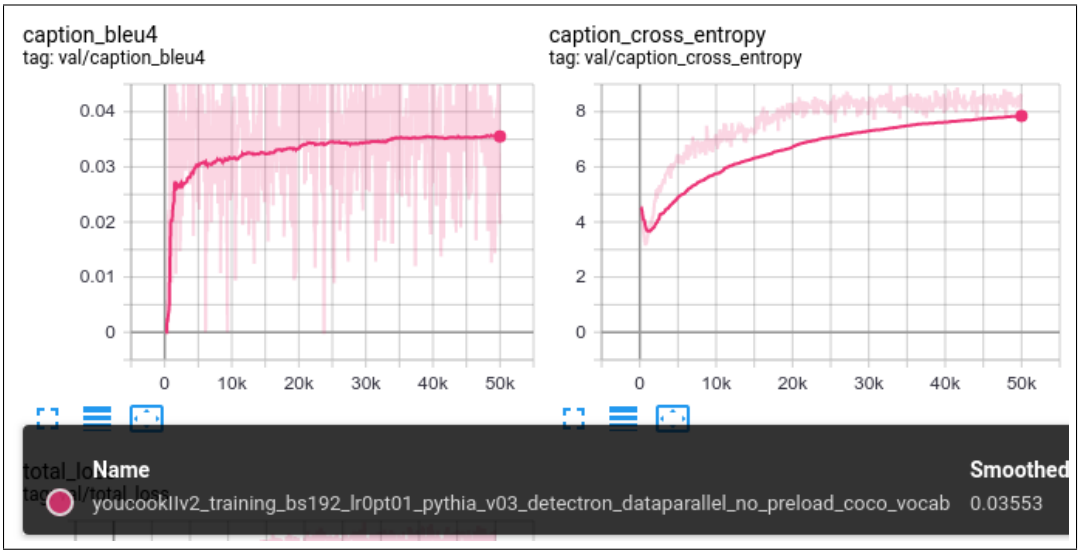

Figure 7.9: Tensorboard visualization of validation iterations.

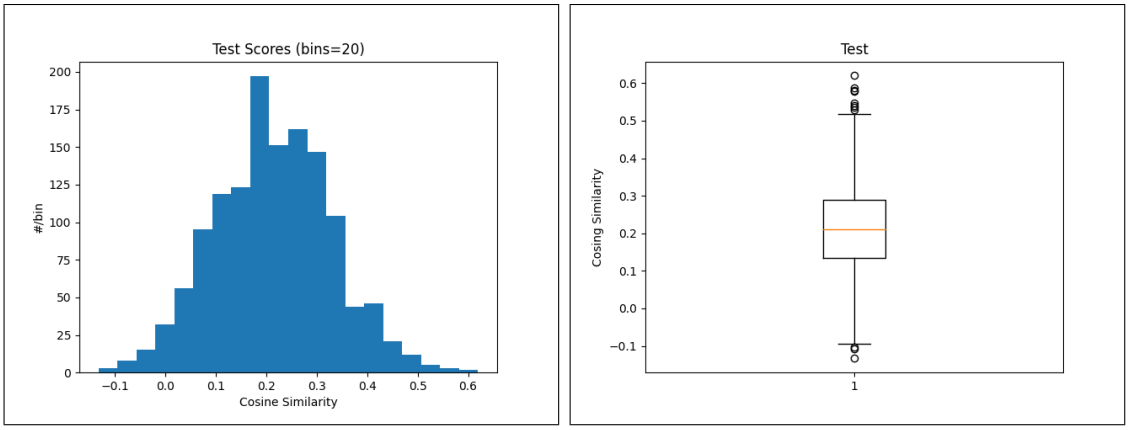

Figure 7.10: Histogram and boxplot of cosine similarity scores from training YouCookII from scratch with default COCO vocabulary.

7.4 Train Pythia on YouCookII without Preloading and with YouCookII Vocabulary

For this experiment, we trained YouCookII from scratch with Pythia v0.3, using data parallel, on 3 GPUs, up to 50,000 iterations. We did not preload a model trained on COCO, and we used the YouCookII vocabulary and COCO vocabulary combined. What we are trying to determine with this experiment is whether the expanded vocabulary has any effect or not. It appears as though it does, but only slightly. See the Results section and Table 8.1.

In the tensorboard output, we see a similar trend as the previous experiment with COCO only vocabulary. BLEU4 converges toward an unrealistically high value during training and cross-entropy is minimized to near 0. The validation scores are slightly higher than the previous experiments but validation cross-entropy does not seem to be minimizing correctly. We're not sure whether to trust this result either but as shown in the Results section and Table 8.1, this experiment did have a higher maximum, mean, and BLEU4 score than the previous experiment, which means that the additionally vocabulary did have an impact on training YouCookII from scratch.

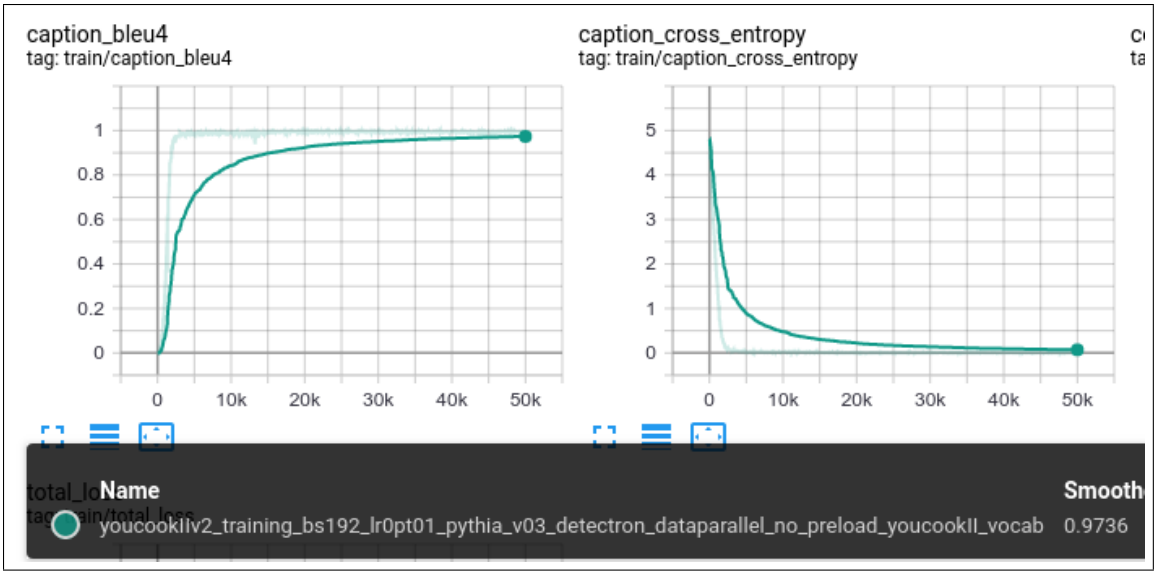

Figure 7.11: Tensorboard visualization of training iterations.

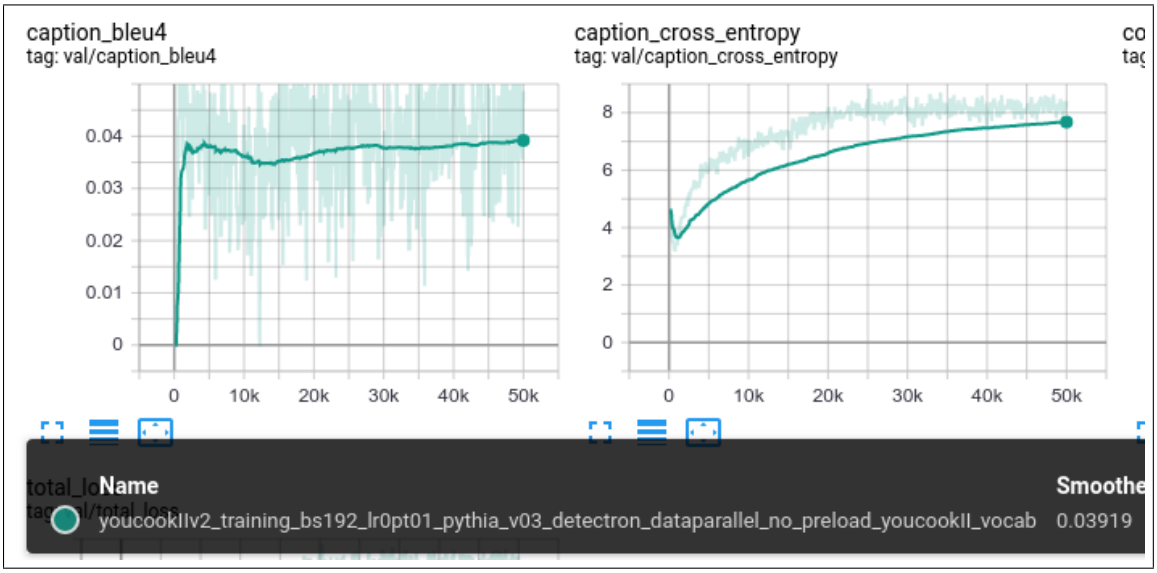

Figure 7.12: Tensorboard visualization of validation iterations.

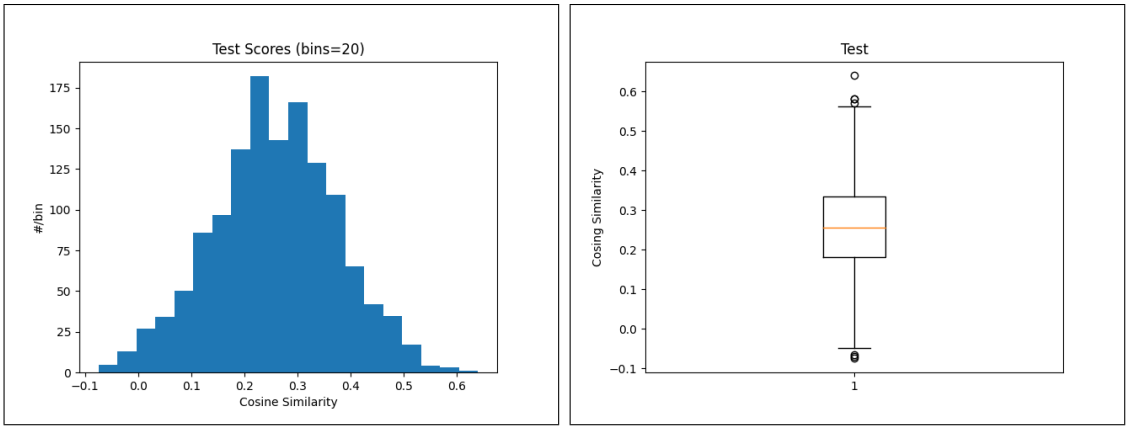

Figure 7.13: Histogram and boxplot of cosine similarity scores from training YouCookII from scratch with YouCookII vocabulary.

7.5 Fine-Tune Train on YouCookII with Preloading and with COCO and YouCookII Vocabulary

For this experiment, we trained COCO from scratch, while expanding the vocabulary to include the YouCookII vocabulary. We used data parallel, on 3 GPUs, up to 50,000 iterations. We then ran fine-tune training on the preloaded model with the YouCookII dataset, using the COCO default vocabulary and the YouCookII vocabulary, combined. Figures 7.14 and 7.15 show the training and validation tensorboard output. Figure 7.16 shows the histograms and boxplots. The final BLEU4 scores was 2.23 and final cross-entropy loss was 21.04.

The tensorboard output shows a similar pattern for training as the previous two experiments: it converges toward an unrealistically high BLEU4 value while cross-entropy is minimized toward near zero. The validation tensorboard output shows a higher BLEU4 validation score than the previous two experiments. The cross-entropy also seems to be maximizing instead of minimizing though, like the previous two experiments. In the Results section and Table 8.1 we can see that this experiment did perform better than either of the previous two experiments with high maximum, mean, and BLEU4, which means that preloading and using a combined vocabulary were beneficial. However, these numbers still do not outperform COCO training without YouCookII data.

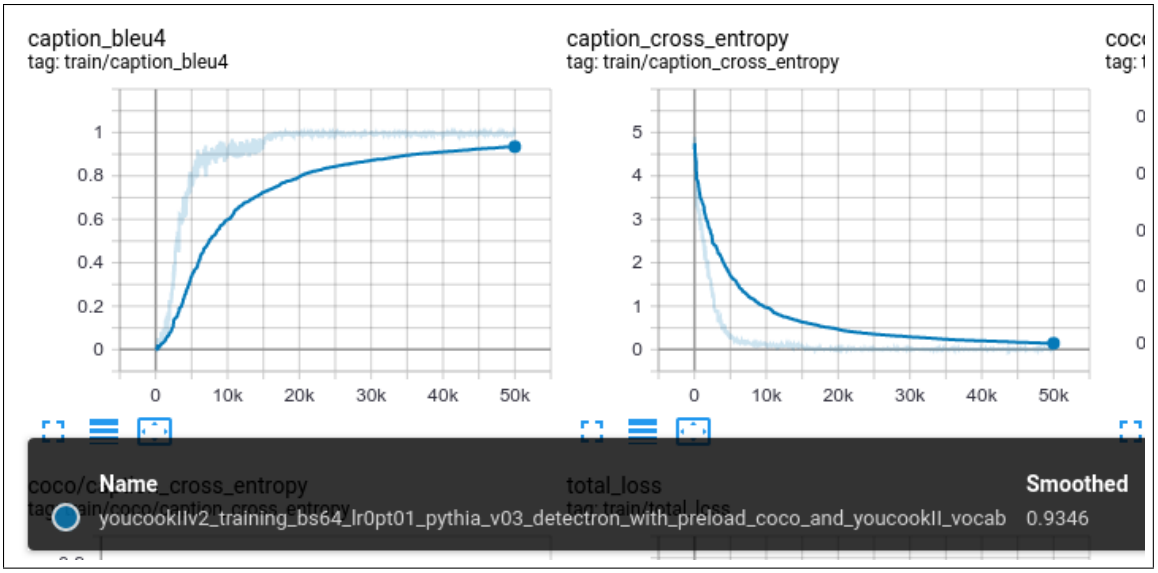

Figure 7.14: Tensorboard visualization of training iterations.

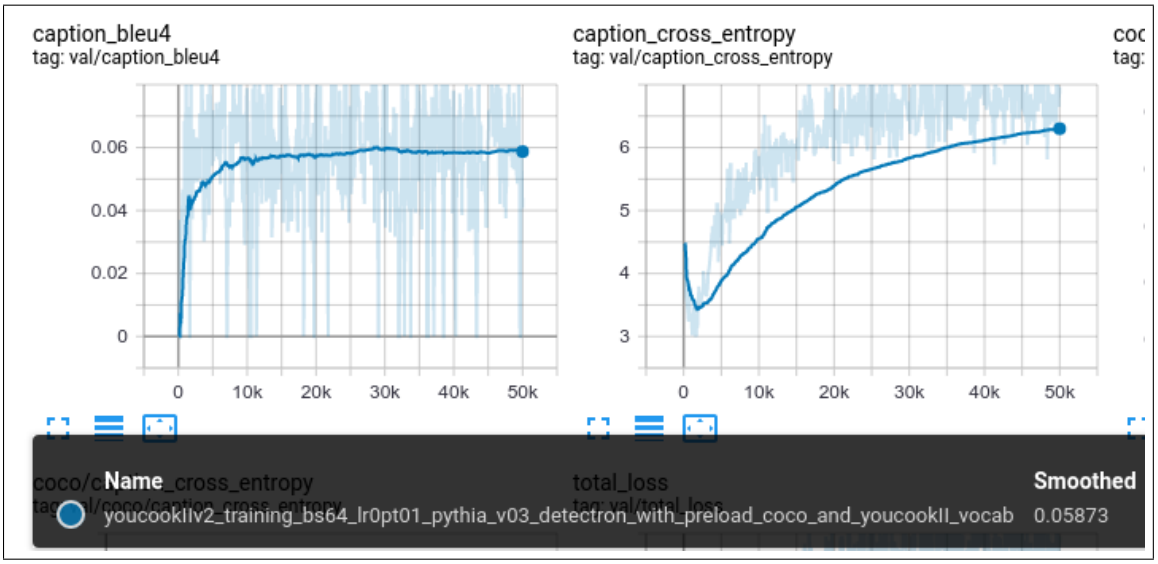

Figure 7.15: Tensorboard visualization of validation iterations.

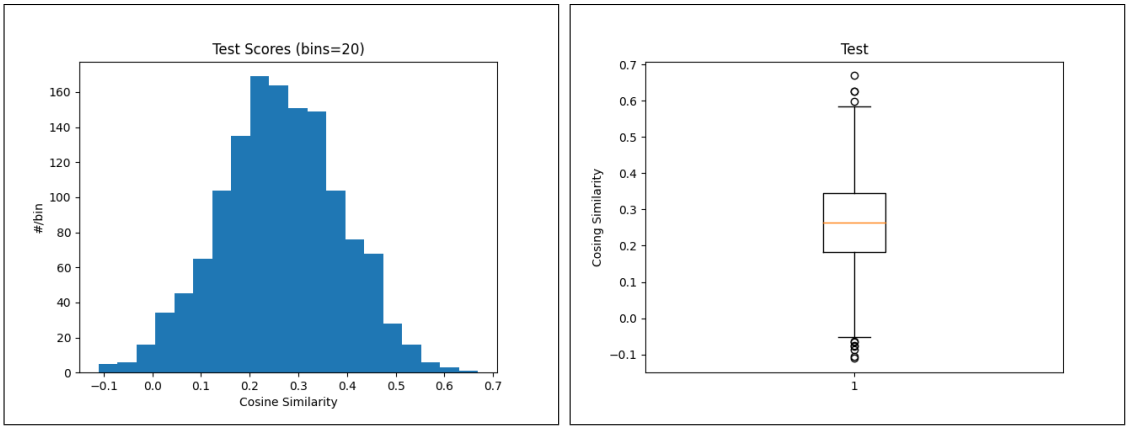

Figure 7.16: Histogram and boxplot of cosine similarity scores from Fine-Tune training YouCookII from scratch with COCO and YouCookII vocabulary.

7.6 Train Pythia on COCO with COCO and YouCookII Vocabulary 1 Caption

The loss curves of previous three experiments show the "hockey-stick" characteristic of declining and sharply shootting up again, rapidly. This is indicitive of overfitting the data. We thought this may be because of the lack of multiple reference sentences in the training set. Recall that YouCookII only has one reference ground-truth sentence per image but COCO has five reference sentences. In order to see if the number of references has an impact on the results, we ran COCO training from scratch again, with the combined vocabulary, but while restricting the number of references to one so that it is more like the YouCookII dataset. You can see the training and validation, BLEU4 and loss graphs in Figures 7.17 and 7.18. Figure 7.19 shows the distribution of cosine similarity scores. The final BLEU4 score was 1.05 and final cross-entropy loss was 20.93.

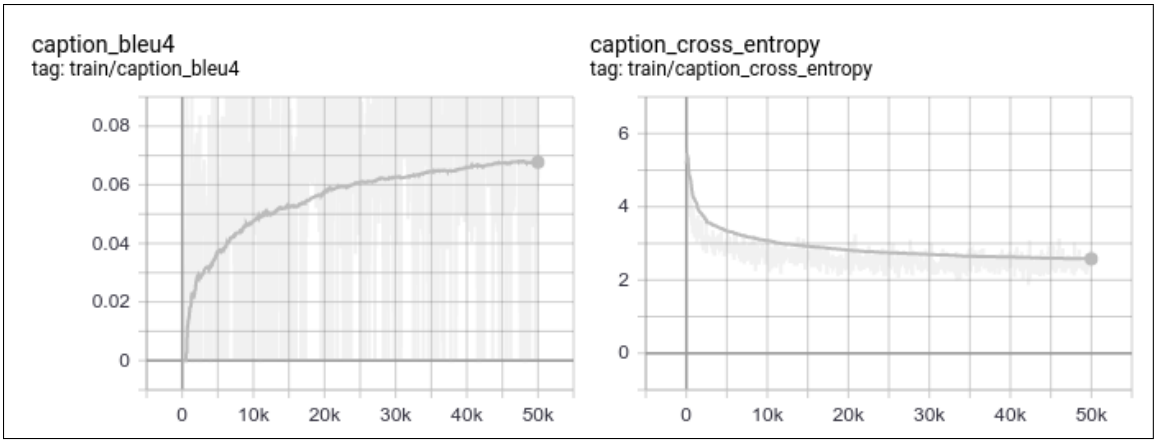

Figure 7.17: Tensorboard visualization of training iterations.

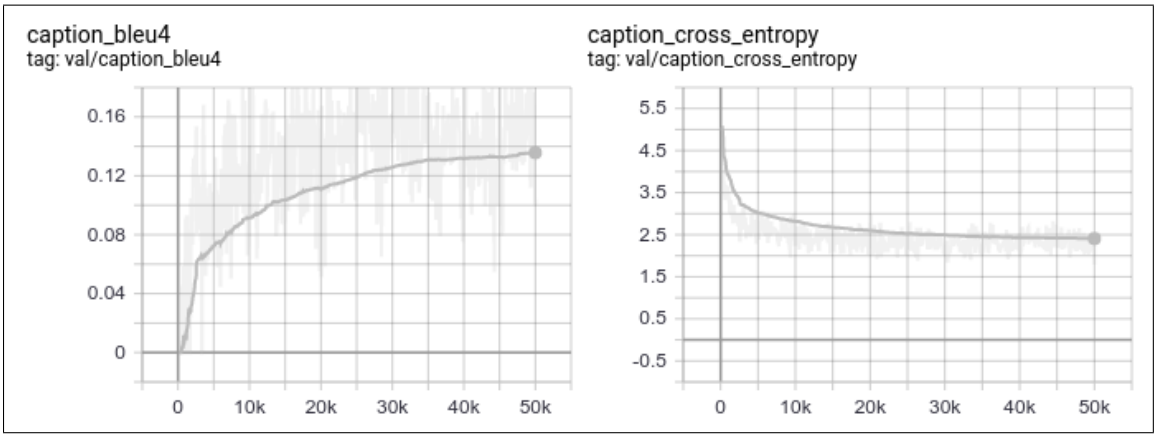

Figure 7.18: Tensorboard visualization of validation iterations.

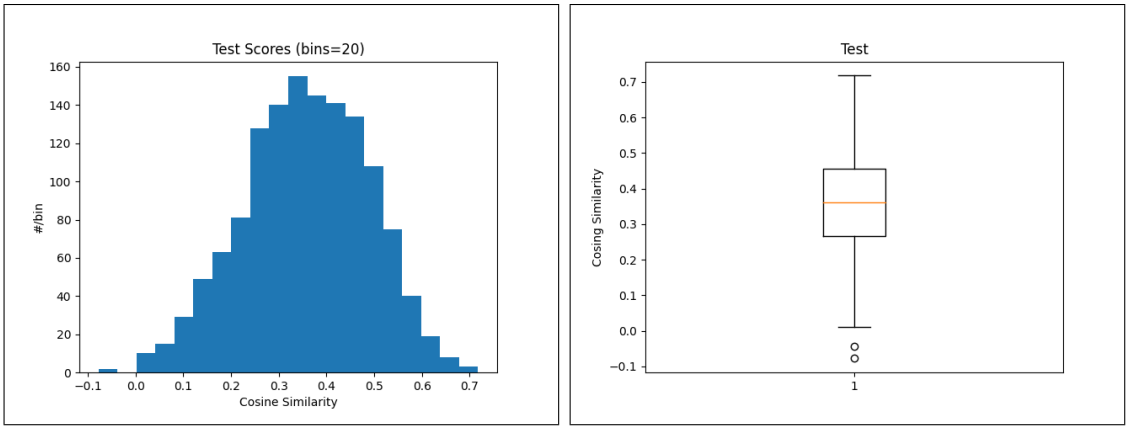

Figure 7.19: Histogram and boxplot of cosine similarity scores COCO training from scratch with COCO and YouCookII vocabulary but only using one reference sentence from the default ground-truth references.

These graphs did not show the "hockey-stick" pattern in the loss graphs, but the final BLEU4 score was incredibly low, very similary to the YouCookII trainings. This shows that a single reference caption does not contain enough information to generalize sufficiently. However, when we run caption generation and cosine similarity scoring, the scores still performed relatively well, and still better than YouCookII training. This is probably because the COCO dataset is much larger than the YouCookII dataset.

7.7 Fine-Tune Train on YouCookII with Preloading and with COCO and YouCookII Vocabulary with 5 keyframes

As a final follow-up experiment, we also hypothesizied that if we are overfitting our data and if we only have one reference caption from the YouCookII ground-truth sentences, then perhaps another way to increase the dataset is use several keyframes from each segment, along with the single caption to train the network. This, however, meant we had a create a new dataset and so it cannot be considered an exact head-to-head comparison with the other experiments. Additionally, we did not shuffle frames acrros subsets: frames from each subset of videos appear in only that subset and do not mix across subsets.

The final results do show a big improvement over the previous BLEU4 scores for the YouCookII experiments, which is encouraging and shows that add more data in this way helps with the overall results of our measured metrics. Figure 7.20 and 7.21 show the training and validation tensorboard BLEU4 and cross-entropy loss graphs. We do not see the "hockey-stick" pattern and it looks like we are fitting the data well. Figure 7.22 shows the distribution of cosine similarity scores on the test set. The final BLEU4 score on the test set was 6.24 and final cross-entropy loss as 24.29.

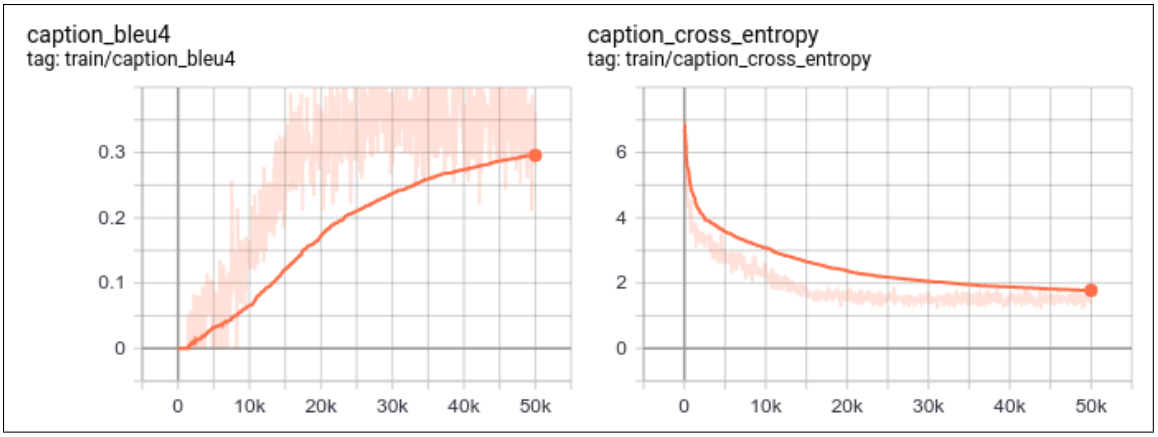

Figure 7.20: Tensorboard visualization of training iterations.

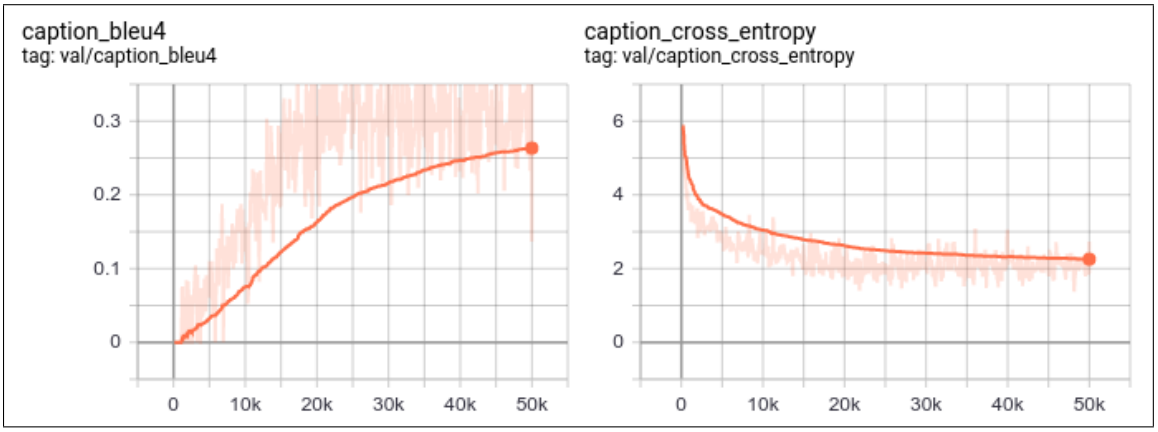

Figure 7.21: Tensorboard visualization of validation iterations.

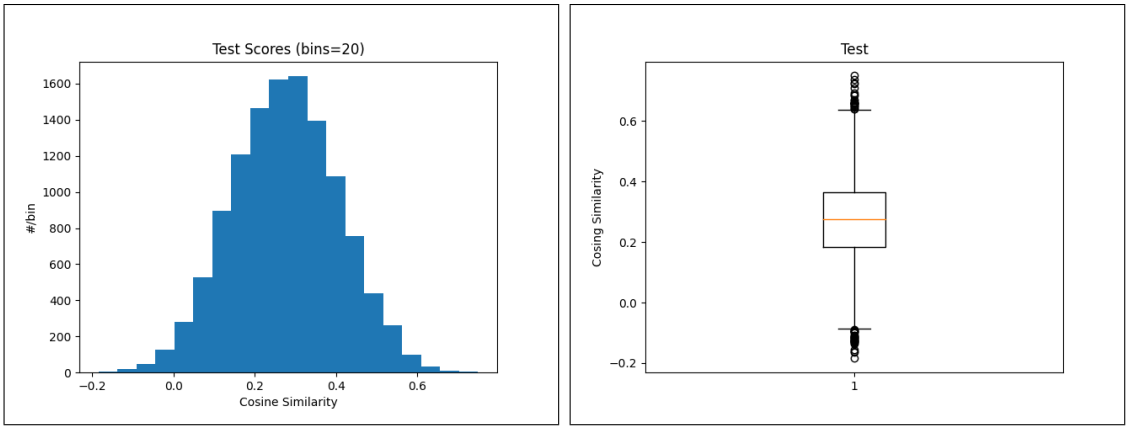

Figure 7.22: Histogram and boxplot of cosine similarity scores for fine-tune training of YouCookII dataset with COCO and YouCookII vocabulary and using 5 keyframes per segment.

Chapter 8

Results

The final results of all experiments are presented in Table 8.1. The best mean cosine-similarity scores come from training COCO from scratch without the YouCookII data. Expanding the vocabulary during COCO training does not seem to have a significant impact on those runs and it shouldn't because those words are not in the COCO images. The interesting thing in this table is that the experiments using the YouCookII data: with COCO vocabulary, with YouCookII vocabulary, and with COCO and YouCookII vocabulary, combined, show progressively better performance, respectively. This is encouraging and makes it seem as though our hypothesis has at least some merit.

The two follow-up experiments are also encouraging. Training COCO from scratch with only one reference sentence shows performance similar to the YouCookII. Expanding the dataset to 5 keyframes in the YouCookII training also improved the training curves and final BLEU4 score significantly. This is encouraging because it

is relatively easy to expand the dataset by adding more keyframes from a segment but more difficult to expand the dataset by adding more ground-truth captions. To get the final word on the performanc we must look at the images and generated captions in the Appendix and see if they correspond with human understanding.

Experiment	Min	Max	Mean±Std	BLEU4	Loss
Baseline	0.1705	0.6123	0.3298±0.0096	—	—
COCO Train COCO Vocab	0.0080	0.6950	0.3702±0.0178	31.97	20.96
COCO Train COCO+YC Vocab	-0.0555	0.7393	0.3700±0.0173	31.47	20.88
YC Train COCO Vocab	-0.1333	0.6200	0.2138±0.0134	0.93	27.49
YC Train YC Vocab	-0.0747	0.6404	0.2575±0.0135	1.81	16.36
YC FT COCO+YC Vocab	-0.1111	0.6693	0.2634±0.0155	2.23	21.04
COCO Train 1 Caption	-0.0777	0.7178	0.3571±0.0171	1.05	20.93
YC FT 5 Frames	-0.1845	0.7496	0.2754 ±0.0171	6.24	24.29

Table 8.1: Minimum, maximum, mean, standard deviation of cosine similarity scores, BLEU4 scores and cross-entropy loss for all experiments on YouCookII test set.

Chapter 9

Challenges and Findings

There was a significant learning curve in using the Pythia framework and integrating a new dataset into it. Using the Pythia framework for this study was in some ways very beneficial because it allowed us to complete the experiments in the way in which we envisioned them. However, a lot of time was taken in learning the framework: making modules and classes, preprocessing data to be the format that Pythia expected, and then validating whether it was working correctly or not. This involved collaboration with colleagues from Georgia Tech as well as collaboration through the Pythia github with developers without whose help this would not have been completed. Everyone was very supportive and helpful and we are grateful for that support.

We can't help but wonder what this project would have been like without using the Pythia framework though. If we did not, we might have had a more thorough understanding of how to design the neural network architectures from scratch. How-

ever, if we took a completely from scratch approach, we may not have been able to run all of these experiments to completion. In the end, we feel that using the Pythia framework ended-up being a very good choice, none-the-less.

Some other challenges were in procuring the entire YouCookII dataset. Since the data is only available as precomputed features, to get access to the raw videos we needed to download each video one at a time from YouTube. Many were unavailable and downloading was interrupted multiple times. This took extra time which could have been spent on the study. In the end, we were able to procure a complete dataset through help from the dataset publishers at University of Michigan.

Other hurdles involved compute, space, and bandwidth constraints. We had access to a 3 GPU system, but training still took a lot of time to train, and then transfer the model to another system for inference, and transfer log output to another system for visualization. Several long running experiments would stop prematurely due to running out of disk space on the host system. And transferring datasets over old DSL lines was a pain point a few times throughout this work.

Chapter 10

Discussion

The actual results of our experiment should be considered inconclusive. We completed a transfer learning experiment in a several ways but did not see the performance increase in indirect metrics (BLEU4 and cosine-similarity) which we had hoped. This does not mean that that idea will not work. On the contrary, it actually throws light on other aspects of this experimental procedure which can be modified and which might yield better results.

The use of cosine-similarity as a guiding metric is probably one of the most interesting things about this study. Since all of the NLP metrics are focused on language tasks and not vision and language tasks it seemed useful and still seems useful to have a metric which works in the joint-embedding space. This metric, however, using the VSE++ code which was pretrained on COCO and default COCO vocab and captions, might be misleading and may thrown-off the results because our keyframes were extracted based off the cosine-similarity score of frame and ground-truth cap-

tion, according to the COCO hypersphere. Perhaps better keyframe extraction can be made by pretraining VSE++ on the expanded vocabulary, as well.

Our method for keyframe extraction is not the only way we could have extracted keyframes from segments. We could have taken the first frame in a segment, the last frame, the middle frame, or randomly sample a frame. Using cosine similarity was just one way. This idea can also extend to taking several keyframes for a segment and training a multi-dimensional vision network with a stack of images. This will be more costly to run but should improve performance.

Using the cosine similarity gave us a way to measure distance in a shared visual-linguistic embedding space. This idea in itself is quite useful and can be applied to other multi-modal applications. It also is presumed, that if applied successfully, could be used to boost performance in a number of applications where other available data is not being used.

The terseness of the ground-truth captions in the YouCookII dataset could also explain some of the performance and would be an area to focus for future work: how to enrich those captions further. Ideas are presented below.

Chapter 11

Conclusions

We completed our task of using transfer learning, and a transfer metric, to fine-tune a pretrained neural network on a domain-specific dataset. While we did not see a performance boost in our metrics, we can still infer that learning new data had occurred, as evident from the experiments of training YouCookII from scratch using default COCO vocabulary, just the YouCookII vocabulary, and the combined COCO and YouCookII vocabulary. The last one being the best and the core experiment supporting our hypothesis.

To get a better feel for how well these improvements translate into actual human understanding, we must now look at the Appendix to see how the images and generated captions side-by-side. One thing that was noted early in this work is the Pythia, trained on COCO, tends to have very similar captions and somewhat redundant. For example, many of them start with "a close up of", "a person", or "a close up of a person". The YouCookII trained captions do not exhibit that pattern

and are more similar to the ground-truth in form but more inaccurate with nouns. This means that there is hope of improving Pythia models in this transfer-learning way, but more work needs to be done.

Overall, we did not see a huge improvement in caption quality but this work is still promising and given more time, could be refined to a better model and experimental procedure.

In the follow-up experiments, expanding the YouCookII dataset to 5 keyframes was found to be beneficial. In the future, this approach will be good to study further.

Chapter 12

Future Work

To extend this work in the future, there are many obvious directions and some not so obvious. A simple idea would be to use different keyframes, either by using a different keyframe extraction scheme or by using the same keyframe scheme but training on the VSE++ cosine similarity service on an expanded vocabulary.

It would be good to have more captions for each image and caption sample. COCO and Flickr datasets both have at least 5 reference captions. We can add more captions by either generate more sentences using Pythia, GLACNet and other caption generators, or by having humans annotate the images.

We could also add more images from each segment to the sample with the caption, then either train on multiple image and caption pairs or design the network to accept a stack of images on the inputs. This would naturally lend well toward developing full video integration.

Additionally cooking datasets may also be used to further enhance the domain

knowledge for transfer learning. There are several good candidate cooking datasets available with Recipe 1 Million+ being a very attractive. We did not have time to explore more datasets for this project, but experimenting with them in the fashion outlined here, may lead to improved results.

As we saw in the follow-up experiments, adding more keyframes is beneficial, but we assume adding a diversity of captions would be even better. The next dataset that we use for transfer learning should have many more reference ground-truth sentences. We believe that this will lead to greater generalization ability in the generated captions.

Using other model architectures may help but we did not get to go into this for this work. Pythia is actively under development and changing the name to MMF (for Mulit-Modal Framework). There are tickets against the development github that are working on integrating BERT into the framework. Others on the mailing list are also interested in using other backbone networks. It looks as though more tasks and datasets for computer vision and NLP problems will be available through the MMF framework in the future.

One thing we did not do here was run each experiment 5 times. It is possible that due to subset selection, we may have hit upon some false positive results. Running the experiments multiple times and taking averages would avoid this.

Reinforcement learning is showing a lot of promise and is gaining a lot of attention. The ViL-BERT [71] and SC-RANK [119] papers show caption generation

through RL, which is where we originally wanted to go with this work but did not have time. We would definitely pick-up here if we continue this direction of research.

The YouCookII project also provides bounding-box information for the videos which did not use but would incorporate in future work.

Once these results are refined more, a follow-up survey should be completed with the describers to see if these captions help with annotating videos.

Appendix

Ground Truth: add the cooked mutton with stock and spices and mix everything well (0.334)
COCO Train, COCO Vocab: a close up of a pan of food on a stove (0.507)
COCO Train, COCO+YC Vocab: a close up of a cat in a pot (0.355)
YC Train, Coco Vocab: add some garlic to the pan (0.125)
YC Train, YC Vocab: add the onions and stir to the pan (0.518)
YC FT, COCO+YC Vocab: add the chicken to the pan (0.337)

Ground Truth: mix milk and an egg with the flour (0.377)
COCO Train, COCO Vocab: an image of a wave on a beach (0.155)
COCO Train, COCO+YC Vocab: a close up of a beach with a green and white sky (0.129)
YC Train, Coco Vocab: place the rice mixture on the center of the wrapper (0.171)
YC Train, YC Vocab: place the filling in the center of the rice and roll it (0.206)
YC FT, COCO+YC Vocab: place the egg on the tortilla (0.301)

Ground Truth: grill the hot dog (0.461)
COCO Train, COCO Vocab: a close up of a plate of food with food (0.230)
COCO Train, COCO+YC Vocab: a close up of a hot dog on a plate (0.356)
YC Train, Coco Vocab: add the onions and garlic to the pan (0.061)
YC Train, YC Vocab: fry the onions and add chopped green onions and fry (0.230)
YC FT, COCO+YC Vocab: add the bacon and stir to the chicken (0.228)

Ground Truth: spread the potatoes on top (0.214)
COCO Train, COCO Vocab: a red bowl filled with meat and vegetables (0.344)
COCO Train, COCO+YC Vocab: a bowl of food with a red bowl of sauce (0.379)
YC Train, Coco Vocab: add the tomatoes to the soup and stir (0.066)
YC Train, YC Vocab: add some tomatoes to the soup and mix it with the vegetables (0.167)
YC FT, COCO+YC Vocab: add the beef and stir (0.146)

Ground Truth: add oil chopped onion bell pepper mushrooms and garlic to a pan (0.380)
COCO Train, COCO Vocab: a pot of food that is on a stove (0.554)
COCO Train, COCO+YC Vocab: a bowl of food is sitting on a stove (0.581)
YC Train, Coco Vocab: add baby water and add shrimp to pan and stir (0.056)
YC Train, YC Vocab: add chopped tomatoes and stir to the pan (0.469)
YC FT, COCO+YC Vocab: add the vegetables and stir to the pan (0.378)

Ground Truth: add the bacon to the bread slices (0.300)
COCO Train, COCO Vocab: a close up of a plate of food on a table (0.163)
COCO Train, COCO+YC Vocab: a table with a sandwich and a sandwich on it (0.100)
YC Train, Coco Vocab: add some lettuce cucumber carrot onions and mushrooms slice and cilantro (0.039)
YC Train, YC Vocab: add tomato tomato tomato tomato tomato tomato and tomato tomato on the pizza and serve (0.303)
YC FT, COCO+YC Vocab: add tomatoes and slices of tomato and bacon (0.400)

Ground Truth: sprinkle green onions and salt (0.217)
COCO Train, COCO Vocab: a pan of food that is on a stove (0.530)
COCO Train, COCO+YC Vocab: a pan of food is sitting on a stove (0.506)
YC Train, Coco Vocab: add chopped cabbage and red wine and to the wok (0.302)
YC Train, YC Vocab: add rice and stir to the soup (0.260)
YC FT, COCO+YC Vocab: add the potatoes and mix well (0.138)

Ground Truth: mix in an egg sauce and beer (0.289)
COCO Train, COCO Vocab: a cake sitting on top of a counter top (0.388)
COCO Train, COCO+YC Vocab: a cake is on a counter top with a spoon (0.381)
YC Train, Coco Vocab: add the stock and water to the macaroni (0.131)
YC Train, YC Vocab: add some lemon juice and a little salt to the bowl and mix it with the mixture (0.181)
YC FT, COCO+YC Vocab: add some sugar and some sugar and a piece of bread with a sauce and a sauce (0.231)

Ground Truth: add feta cheese basil and cherry tomatoes to the eggs (0.203)
COCO Train, COCO Vocab: a bowl of salad and a knife on a table (0.397)
COCO Train, COCO+YC Vocab: a bowl of soup with a spoon and a fork (0.478)
YC Train, Coco Vocab: add the tomatos to the sauce and stir (0.253)
YC Train, YC Vocab: add the tomatoes and chopped tomatoes to the bowl (0.328)
YC FT, COCO+YC Vocab: add the soup to the bowl (0.473)

Ground Truth: take a pinch of dough and roll and put them on a plate (0.261)
COCO Train, COCO Vocab: a close up of a child eating food (0.283)
COCO Train, COCO+YC Vocab: a baby is eating food in a bowl (0.339)
YC Train, Coco Vocab: add sauce and mix to the salad (0.229)
YC Train, YC Vocab: mix the eggs and parmesan cheese on the dough (0.516)
YC FT, COCO+YC Vocab: add the chicken to the bowl (0.241)

Ground Truth: cut small pieces of the meat diagonally against the grain mark (0.217)
COCO Train, COCO Vocab: a person cutting a piece of paper on a table (0.472)
COCO Train, COCO+YC Vocab: a person cutting a knife on a cutting board (0.423)
YC Train, Coco Vocab: add avocado to the tuna (0.114)
YC Train, YC Vocab: cut the onion into a pieces (0.162)
YC FT, COCO+YC Vocab: add some chopped garlic and mix with a knife (0.280)

Ground Truth: wash the rice until the water is clear (0.288)
COCO Train, COCO Vocab: a close up of a person holding a spoon (0.210)
COCO Train, COCO+YC Vocab: a close up of a plate of food with a (0.121)
YC Train, Coco Vocab: add some water to the pan and stir (0.161)
YC Train, YC Vocab: add oil to the pan (0.212)
YC FT, COCO+YC Vocab: add some milk and a pan of water to the pan (0.229)

Ground Truth: peel the skin from the onion and cut the onion into thin slices (0.268)
COCO Train, COCO Vocab: a person cutting up an apple on a cutting board (0.577)
COCO Train, COCO+YC Vocab: a person cutting a green apple with a knife (0.431)
YC Train, Coco Vocab: chop some bell pepper and add to the bowl (0.270)
YC Train, YC Vocab: chop up the onion and add to the pot (0.223)
YC FT, COCO+YC Vocab: chop some celery and cut the carrot (0.261)

Ground Truth: fry the potstickers in the pan (0.565)
COCO Train, COCO Vocab: a plate of food on a stove top (0.468)
COCO Train, COCO+YC Vocab: a pan with a slice of pizza on it (0.454)
YC Train, Coco Vocab: spread the bread on the pan (0.448)
YC Train, YC Vocab: add the tofu to the pan (0.345)
YC FT, COCO+YC Vocab: add slices of bread and slice of bread and slice of bread and slice of bread (0.028)

Ground Truth: add 1 tbsp chili powder and salt to it and whisk everything to prepare a marinate (0.175)
COCO Train, COCO Vocab: a bowl filled with food on top of a table (0.296)
COCO Train, COCO+YC Vocab: a bowl of food is sitting on a table (0.285)
YC Train, Coco Vocab: mix flour corn flour and (0.417)
YC Train, YC Vocab: add salt and pepper to the bowl (0.118)
YC FT, COCO+YC Vocab: add the soup to a bowl (0.337)

Ground Truth: cook the vegetables in the pan (0.364)
COCO Train, COCO Vocab: a person putting macaroni and cheese on a stove (0.471)
COCO Train, COCO+YC Vocab: a person is eating food in a kitchen (0.332)
YC Train, Coco Vocab: add the to the pan (0.282)
YC Train, YC Vocab: add the cooked vegetables to the pan (0.280)
YC FT, COCO+YC Vocab: add the chicken and stir (0.036)

Ground Truth: add sugar vinegar and garlic to the pan (0.370)
COCO Train, COCO Vocab: a man stirring a pot of food on a stove (0.655)
COCO Train, COCO+YC Vocab: a man is cooking in a pan on a stove (0.670)
YC Train, Coco Vocab: add the peas to the pan and stir (0.276)
YC Train, YC Vocab: add the peas to the pan (0.300)
YC FT, COCO+YC Vocab: add the pan to the pan and stir (0.432)

Ground Truth: add oil and mustard to the salad (0.178)
COCO Train, COCO Vocab: a close up of some food on a table (0.245)
COCO Train, COCO+YC Vocab: a person is cutting a bowl of food (0.395)
YC Train, Coco Vocab: add the tomatoes to the pot (0.098)
YC Train, YC Vocab: add chopped tomatoes chopped tomatoes and chopped onions and mix it well (0.270)
YC FT, COCO+YC Vocab: add the salad to the salad (0.176)

Ground Truth: add ginger and garlic to the wok and stir (0.339)
COCO Train, COCO Vocab: a close up of a person cutting up vegetables (0.378)
COCO Train, COCO+YC Vocab: a person cutting a carrot with a knife (0.180)
YC Train, Coco Vocab: add the carrot to the wok and stir (0.403)
YC Train, YC Vocab: add the vegetables and the vegetables and stir (0.284)
YC FT, COCO+YC Vocab: add the vegetables to the pot (0.352)

Ground Truth: add red bell pepper green onion and garlic to a hot pan and stir (0.279)
COCO Train, COCO Vocab: a person cooking food in a pan on a stove (0.588)
COCO Train, COCO+YC Vocab: a person is cooking in a pan on a stove (0.580)
YC Train, Coco Vocab: add some ginger and carrots to the pan (0.285)
YC Train, YC Vocab: add carrots to the pan (0.394)
YC FT, COCO+YC Vocab: add the vegetables to the pan (0.397)

Ground Truth: pour the broth into a bowl with the noodles egg and onion (0.283)
COCO Train, COCO Vocab: a bowl of soup sitting on top of a table (0.481)
COCO Train, COCO+YC Vocab: a bowl of soup with a glass of water (0.545)
YC Train, Coco Vocab: add some lemon juice tomato paste and some fish sauce (0.082)
YC Train, YC Vocab: add cabbage and green onion and mix it with a fork (0.327)
YC FT, COCO+YC Vocab: add the soup to the soup (0.312)

Ground Truth: knead the dough well and make into a sphere (0.416)
COCO Train, COCO Vocab: a close up of an apple on a table (0.134)
COCO Train, COCO+YC Vocab: a close up of a bowl of fruit on a table (0.071)
YC Train, Coco Vocab: add the clams to the pan (0.105)
YC Train, YC Vocab: add onions and green onions to the pan (0.318)
YC FT, COCO+YC Vocab: add the onion and the onion and stir (0.137)

Ground Truth: pour the meat back into the pan and add flour (0.213)
COCO Train, COCO Vocab: a pot of food on a stove top (0.676)
COCO Train, COCO+YC Vocab: a pan of food with a pot of food (0.616)
YC Train, Coco Vocab: add red bell pepper and clams to the pan and stir (0.273)
YC Train, YC Vocab: add the peas and vegetables to the pan and stir (0.362)
YC FT, COCO+YC Vocab: add the vegetables and stir and stir (0.164)

Ground Truth: coat the veal in flour the egg mixture (0.352)
COCO Train, COCO Vocab: a close up of a bowl of food in a kitchen (0.154)
COCO Train, COCO+YC Vocab: a white and white plate with a bottle of food (0.313)
YC Train, Coco Vocab: add the pork and to the eggs and mix (0.347)
YC Train, YC Vocab: add salt to the wok (0.172)
YC FT, COCO+YC Vocab: add the chicken to the bowl (0.223)

Ground Truth: place tuna on top of the roll (0.313)
YC FT, COCO+YC 5 Frames: a piece of chicken with bacon on top of it (0.319)

Ground Truth: wrap them up to pierogi shape (0.177)
YC FT, COCO+YC 5 Frames: a hot dog with a little bit of butter (0.089)

Ground Truth: add mustard seeds and chana dal and cook until seeds crack (0.294)
YC FT, COCO+YC 5 Frames: add oil to the wok and stir (0.194)

Ground Truth: dip the chicken in beaten egg and roll it in cornstarch and set it on a plate (0.252)
YC FT, COCO+YC 5 Frames: add the soup to the bowl and stir (0.189)

Ground Truth: cook the finely chopped cabbage and onion in the same skillet (0.438)
YC FT, COCO+YC 5 Frames: add the vegetables to the salad (0.377)

Ground Truth: pull the cooked beef out of the pan and sprinkle little salt and pepper (0.248)
YC FT, COCO+YC 5 Frames: add the chicken into the pan and stir (0.171)

Ground Truth: transfer some liquid into another bowl to marinate the chicken and let the chicken and veggies marinate for an hour (0.375)
YC FT, COCO+YC 5 Frames: a bowl of pasta and soup to a bowl of soup (0.234)

Ground Truth: add the onion bell pepper and bean sprout to the wok (0.390)
YC FT, COCO+YC 5 Frames: add the vegetables to the pan and stir (0.467)

Ground Truth: place basil leaves in the soup (0.330)
YC FT, COCO+YC 5 Frames: add the vegetables to the pan and stir (0.235)

Ground Truth: add cream to the dal (0.187)
YC FT, COCO+YC 5 Frames: add the chicken to the pan (0.173)

Ground Truth: cook the sandwich in the pan (0.421)
YC FT, COCO+YC 5 Frames: add the meat to the chicken and mix (0.115)

Ground Truth: remove the shrimp from the pan (0.321)
YC FT, COCO+YC 5 Frames: add chicken to the wok and stir (0.351)

Ground Truth: place the patty on the bun (0.437)
YC FT, COCO+YC 5 Frames: add the bread and vegetables to the bread (0.225)

Ground Truth: spread butter on top of escargot (0.226)
YC FT, COCO+YC 5 Frames: add the vegetables to the vegetables and stir (0.111)

Ground Truth: cut the carrots to julienne (0.453)
YC FT, COCO+YC 5 Frames: cut the carrots and add to the bowl (0.451)

Ground Truth: wash the rice until the water is clear (0.288)
YC FT, COCO+YC 5 Frames: add the chicken to the pan and stir (0.073)

Bibliography

[1] https://imdc.ca/completedprojects/livedescribe.

[2] *Audio description for youtube videos*, https://youdescribe.org/.

[3] *Be my eyes - see the world together*, https://www.bemyeyes.com/.

[4] *Blindness statistics — national federation of the blind*, https://www.nfb.org/resources/blindness-statistics.

[5] *Screen reader user survey 7 results*, https://webaim.org/projects/screenreadersurvey7/.

[6] *Tutorial: Adding a dataset*, https://learnpythia.readthedocs.io/en/latest/tutorials/dataset.html.

[7] *Activation function*, May 2020, https://en.wikipedia.org/wiki/Activation_function.

[8] *Bleu*, Jan 2020, https://en.wikipedia.org/wiki/BLEU.

[9] *F1 score*, May 2020, https://en.wikipedia.org/wiki/F1_score.

[10] *N-gram*, May 2020, https://en.wikipedia.org/wiki/N-gram.

[11] *Perplexity*, Feb 2020, https://en.wikipedia.org/wiki/Perplexity.

[12] *Recurrent neural network*, Apr 2020, https://en.wikipedia.org/wiki/Recurrent_neural_network.

[13] *Word error rate*, Feb 2020, https://en.wikipedia.org/wiki/Word_error_rate.

[14] © World Health Organization 2012, ch. GLOBAL DATA ON VISUAL IMPAIRMENTS, 2012, https://www.who.int/blindness/GLOBALDATAFINALforweb.pdf.

[15] Peter Anderson, Basura Fernando, Mark Johnson, and Stephen Gould, *Spice: Semantic propositional image caption evaluation*, European Conference on Computer Vision, Springer, 2016, pp. 382–398.

[16] Peter Anderson, Xiaodong He, Chris Buehler, Damien Teney, Mark Johnson, Stephen Gould, and Lei Zhang, *Bottom-up and top-down attention for image captioning and visual question answering*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 6077–6086.

[17] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh, *Vqa: Visual question answering*, Proceedings of the IEEE international conference on computer vision, 2015, pp. 2425–2433.

[18] Open Assistive, *Magpie*, Jun 2016, https://openassistive.org/item/magpie/.

[19] Andrei Barbu, David Mayo, Julian Alverio, William Luo, Christopher Wang, Dan Gutfreund, Josh Tenenbaum, and Boris Katz, *Objectnet: A large-scale bias-controlled dataset for pushing the limits of object recognition models*, Advances in Neural Information Processing Systems, 2019, pp. 9448–9458.

[20] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool, *Food-101–mining discriminative components with random forests*, European conference on computer vision, Springer, 2014, pp. 446–461.

[21] Boxing Chen and Colin Cherry, *A systematic comparison of smoothing techniques for sentence-level bleu*, Proceedings of the Ninth Workshop on Statistical Machine Translation, 2014, pp. 362–367.

[22] David L Chen and William B Dolan, *Collecting highly parallel data for paraphrase evaluation*, Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies-Volume 1, Association for Computational Linguistics, 2011, pp. 190–200.

[23] Mei Chen, Kapil Dhingra, Wen Wu, Lei Yang, Rahul Sukthankar, and Jie Yang, *Pfid: Pittsburgh fast-food image dataset*, 2009 16th IEEE International Conference on Image Processing (ICIP), IEEE, 2009, pp. 289–292.

[24] Leave Authors Anonymous for Submission City and Country e-mail address, *Human-in-the-loop machine learning to increase video accessibility for visually impaired and blind users*, **20** (2019).

[25] Yin Cui, Guandao Yang, Andreas Veit, Xun Huang, and Serge Belongie, *Learning to evaluate image captioning*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 5804–5812.

[26] Bo Dai, Sanja Fidler, Raquel Urtasun, and Dahua Lin, *Towards diverse and natural image descriptions via a conditional gan*, Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2970–2979.

[27] Abhishek Das, Satwik Kottur, Khushi Gupta, Avi Singh, Deshraj Yadav, José M.F. Moura, Devi Parikh, and Dhruv Batra, *Visual Dialog*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[28] Pradipto Das, Chenliang Xu, Richard F Doell, and Jason J Corso, *A thousand frames in just a few words: Lingual description of videos through latent topics and sparse object stitching*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2013, pp. 2634–2641.

[29] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei, *Imagenet: A large-scale hierarchical image database*, 2009 IEEE conference on computer vision and pattern recognition, Ieee, 2009, pp. 248–255.

[30] Michael Denkowski and Alon Lavie, *Meteor universal: Language specific translation evaluation for any target language*, Proceedings of the ninth workshop on statistical machine translation, 2014, pp. 376–380.

[31] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, *Bert: Pre-training of deep bidirectional transformers for language understanding*, arXiv preprint arXiv:1810.04805 (2018).

[32] Author dprogrammer, *Rnn, lstm & gru*, Apr 2019, `http://dprogrammer.org/rnn-lstm-gru`.

[33] Fartash Faghri, David J Fleet, Jamie Ryan Kiros, and Sanja Fidler, *Vse++: Improving visual-semantic embeddings with hard negatives*, arXiv preprint arXiv:1707.05612 (2017).

[34] Thushan Ganegedara, *Natural language processing with tensorflow: teach language to machines using pythons deep learning library*, Packt Publishing, 2018.

[35] Kavita Ganesan, *Rouge 2.0: Updated and improved measures for evaluation of summarization tasks*, arXiv preprint arXiv:1803.01937 (2018).

[36] Spandana Gella, Mike Lewis, and Marcus Rohrbach, *A dataset for telling the stories of social media videos*, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 968–974.

[37] Ross Girshick, *Fast r-cnn*, Proceedings of the IEEE international conference on computer vision, 2015, pp. 1440–1448.

[38] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik, *Rich feature hierarchies for accurate object detection and semantic segmentation*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2014, pp. 580–587.

[39] Sergio Guadarrama, Niveda Krishnamoorthy, Girish Malkarnenkar, Subhashini Venugopalan, Raymond Mooney, Trevor Darrell, and Kate Saenko, *Youtube2text: Recognizing and describing arbitrary activities using semantic hierarchies and zero-shot recognition*, Proceedings of the IEEE international conference on computer vision, 2013, pp. 2712–2719.

[40] Danna Gurari, Qing Li, Abigale J Stangl, Anhong Guo, Chi Lin, Kristen Grauman, Jiebo Luo, and Jeffrey P Bigham, *Vizwiz grand challenge: Answering visual questions from blind people*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 3608–3617.

[41] Aaron Li-Feng Han, Derek F Wong, Lidia S Chao, Liangye He, Yi Lu, Junwen Xing, and Xiaodong Zeng, *Language-independent model for machine translation evaluation with reinforced factors*, Proceedings of the 14th International Conference of Machine Translation Summit, International Association for Machine Translation Press, 2013, pp. 215–222.

[42] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick, *Mask r-cnn*, Proceedings of the IEEE international conference on computer vision, 2017, pp. 2961–2969.

[43] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, *Deep residual learning for image recognition*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.

[44] Micah Hodosh, Peter Young, and Julia Hockenmaier, *Framing image description as a ranking task: Data, models and evaluation metrics*, Journal of Artificial Intelligence Research **47** (2013), 853–899.

[45] Saihui Hou, Yushan Feng, and Zilei Wang, *Vegfru: A domain-specific dataset for fine-grained visual categorization*, Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 541–549.

[46] Ting-Hao Huang, Francis Ferraro, Nasrin Mostafazadeh, Ishan Misra, Aishwarya Agrawal, Jacob Devlin, Ross Girshick, Xiaodong He, Pushmeet Kohli, Dhruv Batra, et al., *Visual storytelling*, Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2016, pp. 1233–1239.

[47] Hideki Isozaki, Tsutomu Hirao, Kevin Duh, Katsuhito Sudoh, and Hajime Tsukada, *Automatic evaluation of translation quality for distant language pairs*, Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2010, pp. 944–952.

[48] Ahmad Babaeian Jelodar, Md Sirajus Salekin, and Yu Sun, *Identifying object states in cooking-related images*, arXiv preprint arXiv:1805.06956 (2018).

[49] Yu Jiang, Vivek Natarajan, Xinlei Chen, Marcus Rohrbach, Dhruv Batra, and Devi Parikh, *Pythia v0. 1: the winning entry to the vqa challenge 2018*, arXiv preprint arXiv:1807.09956 (2018).

[50] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C Lawrence Zitnick, and Ross Girshick, *Clevr: A diagnostic dataset for compositional language and elementary visual reasoning*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 2901–2910.

[51] Yoshiyuki Kawano and Keiji Yanai, *Foodcam: A real-time food recognition system on a smartphone*, Multimedia Tools and Applications **74** (2015), no. 14, 5263–5287.

[52] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, et al., *The kinetics human action video dataset*, arXiv preprint arXiv:1705.06950 (2017).

[53] Renu Khandelwal, *Word embeddings for nlp*, Dec 2019, `https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4`.

[54] Taehyeong Kim, Min-Oh Heo, Seonil Son, Kyoung-Wha Park, and Byoung-Tak Zhang, *Glac net: Glocal attention cascading networks for multi-image cued story generation*, arXiv preprint arXiv:1805.10973 (2018).

[55] Ryan Kiros, Yukun Zhu, Russ R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler, *Skip-thought vectors*, Advances in neural information processing systems, 2015, pp. 3294–3302.

[56] Dietrich Klakow and Jochen Peters, *Testing the correlation of word error rate and perplexity*, Speech Communication **38** (2002), no. 1-2, 19–28.

[57] Marcus Klasson, Cheng Zhang, and Hedvig Kjellström, *A hierarchical grocery store image dataset with visual and semantic labels*, 2019 IEEE Winter Conference on Applications of Computer Vision (WACV), IEEE, 2019, pp. 491–500.

[58] Satwik Kottur, José MF Moura, Devi Parikh, Dhruv Batra, and Marcus Rohrbach, *Clevr-dialog: A diagnostic dataset for multi-round reasoning in visual dialog*, arXiv preprint arXiv:1903.03166 (2019).

[59] Ranjay Krishna, Kenji Hata, Frederic Ren, Li Fei-Fei, and Juan Carlos Niebles, *Dense-captioning events in videos*, International Conference on Computer Vision (ICCV), 2017.

[60] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton, *Imagenet classification with deep convolutional neural networks*, Advances in neural information processing systems, 2012, pp. 1097–1105.

[61] Hildegard Kuehne, Hueihan Jhuang, Estíbaliz Garrote, Tomaso Poggio, and Thomas Serre, *Hmdb: a large video database for human motion recognition*, 2011 International Conference on Computer Vision, IEEE, 2011, pp. 2556–2563.

[62] Matt Kusner, Yu Sun, Nicholas Kolkin, and Kilian Weinberger, *From word embeddings to document distances*, International conference on machine learning, 2015, pp. 957–966.

[63] Baoli Li and Liping Han, *Distance weighted cosine similarity measure for text classification*, International Conference on Intelligent Data Engineering and Automated Learning, Springer, 2013, pp. 611–618.

[64] Yingbo Li and Bernard Merialdo, *Vert: Automatic evaluation of video summaries*, Proceedings of the 18th ACM International Conference on Multimedia (New York, NY, USA), MM 10, Association for Computing Machinery, 2010, `https://doi.org/10.1145/1873951.1874095`, p. 851854.

[65] Chin-Yew Lin, *Rouge: A package for automatic evaluation of summaries acl*, Proceedings of Workshop on Text Summarization Branches Out Post Conference Workshop of ACL, 2004, pp. 2017–05.

[66] Chin-Yew Lin and Franz Josef Och, *Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics*, Proceedings of the 42nd Annual Meeting on Association for Computational Linguistics, Association for Computational Linguistics, 2004, p. 605.

[67] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick, *Microsoft coco: Common objects in context*, European conference on computer vision, Springer, 2014, pp. 740–755.

[68] Chang Liu, Yu Cao, Yan Luo, Guanling Chen, Vinod Vokkarane, and Yunsheng Ma, *Deepfood: Deep learning-based food image recognition for computer-aided dietary assessment*, International Conference on Smart Homes and Health Telematics, Springer, 2016, pp. 37–48.

[69] Ding Liu and Daniel Gildea, *Syntactic features for evaluation of machine translation*, Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, 2005, pp. 25–32.

[70] Siqi Liu, Zhenhai Zhu, Ning Ye, Sergio Guadarrama, and Kevin Murphy, *Improved image captioning via policy gradient optimization of spider*, Proceedings of the IEEE international conference on computer vision, 2017, pp. 873–881.

[71] Jiasen Lu, Dhruv Batra, Devi Parikh, and Stefan Lee, *Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks*, Advances in Neural Information Processing Systems, 2019, pp. 13–23.

[72] Jiasen Lu, Vedanuj Goswami, Marcus Rohrbach, Devi Parikh, and Stefan Lee, *12-in-1: Multi-task vision and language representation learning*, arXiv preprint arXiv:1912.02315 (2019).

[73] Huaishao Luo, Lei Ji, Botian Shi, Haoyang Huang, Nan Duan, Tianrui Li, Xilin Chen, and Ming Zhou, *Univilm: A unified video and language pre-training model for multimodal understanding and generation*, arXiv preprint arXiv:2002.06353 (2020).

[74] Ruotian Luo, Brian Price, Scott Cohen, and Gregory Shakhnarovich, *Discriminability objective for training descriptive captions*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 6964–6974.

[75] Arjun Majumdar, Ayush Shrivastava, Stefan Lee, Peter Anderson, Devi Parikh, and Dhruv Batra, *Improving vision-and-language navigation with image-text pairs from the web*, arXiv preprint arXiv:2004.14973 (2020).

[76] Javier Marin, Aritro Biswas, Ferda Ofli, Nicholas Hynes, Amaia Salvador, Yusuf Aytar, Ingmar Weber, and Antonio Torralba, *Recipe1m+: A dataset for learning cross-modal embeddings for cooking recipes and food images*, IEEE transactions on pattern analysis and machine intelligence (2019).

[77] Simon Mezgec and Barbara Koroušić Seljak, *Nutrinet: a deep learning food and drink image recognition system for dietary assessment*, Nutrients **9** (2017), no. 7, 657.

[78] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean, *Efficient estimation of word representations in vector space*, arXiv preprint arXiv:1301.3781 (2013).

[79] Hieu V Nguyen and Li Bai, *Cosine similarity metric learning for face verification*, Asian conference on computer vision, Springer, 2010, pp. 709–720.

[80] Shruti Palaskar, Jindrich Libovickỳ, Spandana Gella, and Florian Metze, *Multimodal abstractive summarization for how2 videos*, arXiv preprint arXiv:1906.07901 (2019).

[81] Lili Pan, Samira Pouyanfar, Hao Chen, Jiaohua Qin, and Shu-Ching Chen, *Deepfood: Automatic multi-class classification of food ingredients using deep learning*, 2017 IEEE 3rd international conference on collaboration and internet computing (CIC), IEEE, 2017, pp. 181–189.

[82] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu, *Bleu: a method for automatic evaluation of machine translation*, Proceedings of the 40th annual meeting on association for computational linguistics, Association for Computational Linguistics, 2002, pp. 311–318.

[83] Jeffrey Pennington, Richard Socher, and Christopher D Manning, *Glove: Global vectors for word representation*, Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), 2014, pp. 1532–1543.

[84] Sabina Pokhrel, *Beginners guide to understanding convolutional neural networks*, Sep 2019, https://towardsdatascience.com/beginners-guide-to-understanding-convolutional-neural-networks-ae9ed58bb17d.

[85] Cyrus Rashtchian, Peter Young, Micah Hodosh, and Julia Hockenmaier, *Collecting image annotations using amazon's mechanical turk*, Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk, Association for Computational Linguistics, 2010, pp. 139–147.

[86] Siva Reddy, Danqi Chen, and Christopher D Manning, *Coqa: A conversational question answering challenge*, Transactions of the Association for Computational Linguistics **7** (2019), 249–266.

[87] Michaela Regneri, Marcus Rohrbach, Dominikus Wetzel, Stefan Thater, Bernt Schiele, and Manfred Pinkal, *Grounding action descriptions in videos*, Transactions of the Association for Computational Linguistics **1** (2013), 25–36.

[88] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun, *Faster r-cnn: Towards real-time object detection with region proposal networks*, Advances in neural information processing systems, 2015, pp. 91–99.

[89] Zhou Ren, Xiaoyu Wang, Ning Zhang, Xutao Lv, and Li-Jia Li, *Deep reinforcement learning-based image captioning with embedding reward*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 290–298.

[90] Anna Rohrbach, Marcus Rohrbach, Wei Qiu, Annemarie Friedrich, Manfred Pinkal, and Bernt Schiele, *Coherent multi-sentence video description with variable level of detail*, German conference on pattern recognition, Springer, 2014, pp. 184–195.

[91] Anna Rohrbach, Marcus Rohrbach, and Bernt Schiele, *The long-short story of movie description*, German conference on pattern recognition, Springer, 2015, pp. 209–221.

[92] Anna Rohrbach, Marcus Rohrbach, Niket Tandon, and Bernt Schiele, *A dataset for movie description*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 3202–3212.

[93] Marcus Rohrbach, Sikandar Amin, Mykhaylo Andriluka, and Bernt Schiele, *A database for fine grained activity detection of cooking activities*, 2012 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, 2012, pp. 1194–1201.

[94] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba, *Learning cross-modal embeddings for cooking recipes and food images*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 3020–3028.

[95] John R. Searle, *Minds, brains, and programs*, Behavioral and Brain Sciences **3** (1980), no. 3, 417424.

[96] Naeha Sharif, Lyndon White, Mohammed Bennamoun, and Syed Afaq Ali Shah, *Learning-based composite metrics for improved caption evaluation*, Proceedings of ACL 2018, student research workshop, 2018, pp. 14–20.

[97] Piyush Sharma, Nan Ding, Sebastian Goodman, and Radu Soricut, *Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning*, Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 2556–2565.

[98] Grigori Sidorov, Alexander Gelbukh, Helena Gómez-Adorno, and David Pinto, *Soft similarity and soft cosine measure: Similarity of features in vector space model*, Computación y Sistemas **18** (2014), no. 3, 491–504.

[99] Gunnar A Sigurdsson, Abhinav Gupta, Cordelia Schmid, Ali Farhadi, and Karteek Alahari, *Actor and observer: Joint modeling of first and third-person videos*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7396–7404.

[100] _____, *Charades-ego: A large-scale dataset of paired third and first person videos*, arXiv preprint arXiv:1804.09626 (2018).

[101] Karen Simonyan and Andrew Zisserman, *Very deep convolutional networks for large-scale image recognition*, arXiv preprint arXiv:1409.1556 (2014).

[102] Amanpreet Singh, Vedanuj Goswami, Vivek Natarajan, Yu Jiang, Xinlei Chen, Meet Shah, Marcus Rohrbach, Dhruv Batra, and Devi Parikh, *Pythia-a platform for vision & language research*, SysML Workshop, NeurIPS, vol. 2018, 2018.

[103] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach, *Towards vqa models that can read*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019.

[104] Amanpreet Singh, Vivek Natarjan, Meet Shah, Yu Jiang, Xinlei Chen, Devi Parikh, and Marcus Rohrbach, *Towards vqa models that can read*, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019, pp. 8317–8326.

[105] Matthew Snover, Bonnie Dorr, Richard Schwartz, Linnea Micciulla, and John Makhoul, *A study of translation edit rate with targeted human annotation*, Proceedings of association for machine translation in the Americas, vol. 200, 2006.

[106] Matthew G Snover, Nitin Madnani, Bonnie Dorr, and Richard Schwartz, *Ter-plus: paraphrase, semantic, and alignment enhancements to translation edit rate*, Machine Translation **23** (2009), no. 2-3, 117–127.

[107] Seonil Son, Byoung-Tak Zhang, and VIST GLACNet, *Glacnet+: Towards vivid storytelling by combining visual and dynamic entity representations*.

[108] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah, *Ucf101: A dataset of 101 human actions classes from videos in the wild*, arXiv preprint arXiv:1212.0402 (2012).

[109] Rachael Tatman, *Evaluating text output in nlp: Bleu at your own risk*, Jul 2019, `https://towardsdatascience.com/evaluating-text-output-in-nlp-bleu-at-your-own-risk-e8609665a213`.

[110] Rohit Thakur, *Step by step vgg16 implementation in keras for beginners*, Aug 2019, `https://towardsdatascience.com/step-by-step-vgg16-implementation-in-keras-for-beginners-a833c686ae6c`.

[111] Utah State University, *Center for persons with disabilities*, `https://www.cpd.usu.edu/`.

[112] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin, *Attention is all you need*, Advances in neural information processing systems, 2017, pp. 5998–6008.

[113] Andrea Vedaldi and Brian Fulkerson, *Vlfeat: An open and portable library of computer vision algorithms*, Proceedings of the 18th ACM international conference on Multimedia, 2010, pp. 1469–1472.

[114] Ramakrishna Vedantam, C Lawrence Zitnick, and Devi Parikh, *Cider: Consensus-based image description evaluation*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 4566–4575.

[115] Liwei Wang, Yin Li, Jing Huang, and Svetlana Lazebnik, *Learning two-branch neural networks for image-text matching tasks*, IEEE Transactions on Pattern Analysis and Machine Intelligence **41** (2018), no. 2, 394–407.

[116] Xin Wang, Jiawei Wu, Junkun Chen, Lei Li, Yuan-Fang Wang, and William Yang Wang, *Vatex: A large-scale, high-quality multilingual dataset for video-and-language research*, Proceedings of the IEEE International Conference on Computer Vision, 2019, pp. 4581–4591.

[117] Jun Xu, Tao Mei, Ting Yao, and Yong Rui, *Msr-vtt: A large video description dataset for bridging video and language*, Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 5288–5296.

[118] Semih Yagcioglu, Aykut Erdem, Erkut Erdem, and Nazli Ikizler-Cinbis, *Recipeqa: A challenge dataset for multimodal comprehension of cooking recipes*, arXiv preprint arXiv:1809.00812 (2018).

[119] Shiyang Yan, Yang Hua, and Neil Robertson, *Sc-rank: Improving convolutional image captioning with self-critical learning and ranking metric-based reward*.

[120] Ilmi Yoon, *Human-in-the-loop machine learning to increase video accessibility for visually impaired and blind users*, preprint (2019).

[121] Peter Young, Alice Lai, Micah Hodosh, and Julia Hockenmaier, *From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions*, Transactions of the Association for Computational Linguistics **2** (2014), 67–78.

[122] Kuo-Hao Zeng, Tseng-Hung Chen, Juan Carlos Niebles, and Min Sun, *Title generation for user generated videos*, European conference on computer vision, Springer, 2016, pp. 609–625.

[123] Luowei Zhou, Nathan Louis, and Jason J Corso, *Weakly-supervised video object grounding from text by loss weighting and object interaction*, British Machine Vision Conference, 2018.

[124] Luowei Zhou, Chenliang Xu, and Jason J. Corso, *Youcook2 - towards automatic learning of procedures from web instructional videos*, 2017, `https://arxiv.org/pdf/1703.09788.pdf`.